synapticTrack Consolidated Project Document

Chong Shik Park

2026-08-18

Abstract

This document consolidates the Markdown documentation in the `synapticTrack` repository into a single paper-oriented source. The project author is Chong Shik Park, Department of Accelerator Science and Center for Accelerator Research, Korea University, Sejong, 30019 Republic of Korea. It is organized to support later manuscript writing: project motivation, literature context, physics models, simulation code structure, simulation procedures, analysis methods, analysis procedures, installation, and operating instructions.

Literature Review and Technical Context

`synapticTrack` sits at the intersection of charged-particle beam transport, surrogate modeling, differentiable inference, and reinforcement learning for accelerator controls. Linear transverse beam descriptions follow the standard Courant-Snyder/Twiss formalism for second moments and emittance (Courant and Snyder 1958; Wiedemann 2015). Production sampling uses Sobol low-discrepancy sequences to cover high-dimensional control and beam-state spaces more evenly than independent pseudo-random draws (Sobol' 1967). The surrogate and inference layers are implemented with PyTorch, which provides autograd and tensor execution across CPU/GPU devices (Paszke et al. 2019). The project also uses Python scientific infrastructure for arrays, tabular data, optimization, and machine-learning evaluation (Harris et al. 2020; Virtanen et al. 2020; team 2020; Pedregosa et al. 2011). Planned orbit-correction environments use the Gymnasium API and reinforcement-learning baselines from Stable-Baselines3 (Towers et al. 2024; Raffin et al. 2021). External high-fidelity comparison is anchored by accelerator tracking codes such as TRACK (Ostroumov and Aseev 2006).

The practical objective is to build a staged path from deterministic LEBT dataset generation through multi-output neural surrogates, physics-regularized covariance outputs, beam-state inference, MEBT surrogate modeling, and eventually RL orbit correction and higher-fidelity reconstruction. Current maintained outputs include RAON LEBT and MEBT notebook workspaces, staged no-offset Sobol

production and training notebooks, versioned training artifact manifests, read-only HDF5 compatibility views, publication-ready metric tables, LINAC 2026 paper assets, and a generated static documentation site without `polyfill.io` dependencies.

Project Overview

Source Markdown: `README.md`

synapticTrack

synapticTrack is a modular, extensible Python framework for accelerator optimization using **machine learning** techniques.

Purpose The framework is currently under active development to support **orbit correction and optimization** in **heavy-ion linear accelerators**, specifically focused on the **LEBT (Low Energy Beam Transport)** section.

Features (Planned & In Progress)

- ML-based orbit correction using simulation and diagnostic data
- Interoperability with multiple beam dynamics codes:
  - TRACK (Fortran)
  - JuTrack (Julia)
- Support for:
  - Multi-ion species (different A/Q)
  - Variable charge states and currents
  - Simulation-to-reality generalization

Repository Structure The repository now contains the package code, RAON lattice/notebook workspaces, example workflows, paper assets, and generated model-output areas. Generated run products are intentionally kept separate from reusable source modules.

```
synapticTrack/
+-- .github/workflows/          # GitHub Actions CI definitions
+-- RAON/                       # RAON-specific lattice files and notebooks
|   +-- LEBT_lattice/           # LEBT TRACK lattice/input files
|   +-- LEBT_notebooks/         # LEBT calibration, orbit-correction, and extended-model not
|   +-- MEBT_lattice/           # MEBT TRACK lattice/input files
|   +-- MEBT_notebooks/         # MEBT lattice creation/loading and segmented TRACK run note
+-- TRACK/                      # Local TRACK-related runtime assets, when present
+-- bin/                        # Command-line helper scripts
+-- configs/                    # Dataset, hardware, LEBT, local, and ML configuration files
+-- data/                       # Processed/shared data products
+-- docs/                       # Sphinx docs, task notes, paper material, and conference sc
```

```
|   +-- codex_tasks/            # Implementation task specifications and audit notes
|   +-- linac2026/              # LINAC 2026 JACoW paper source, bibliography, and figures
|   +-- paper/                  # Paper/report helper utilities
|   +-- site/                   # Documentation/site helper utilities
+-- examples/                   # Worked examples and legacy/reference notebooks
|   +-- analysis/               # Scanner and Twiss-analysis examples
|   +-- beam/                   # Beam object examples
|   +-- jutrack/                # JuTrack integration examples
|   +-- lebt_orbit_corrections/ # LEBT calibration, surrogate, and RL orbit-correction examp
|   +-- visualization/          # Plotting examples
+-- models/                     # Generated training outputs, summaries, comparisons, and me
+-- notebooks/                  # Main staged LEBT production and model-training notebooks
+-- run.production/             # Large staged production simulation outputs
+-- scripts/                    # Repository maintenance and notebook hygiene scripts
+-- src/synapticTrack/          # Installable Python package
|   +-- analysis/               # Scanner and diagnostic analysis helpers
|   +-- beam/                   # Beam, Gaussian, scanner, DataFrame, and Twiss utilities
|   +-- cli_commands/           # CLI preflight, TRACK checks, and validation commands
|   +-- data/                   # Dataset schema, input/target views, batch I/O, and audits
|   +-- hardware/               # Steering calibration and actuator abstractions
|   +-- inference/              # Beam-state inference interfaces, losses, objectives, and c
|   +-- io/                     # TRACK, JuTrack, FLAME, OPAL, scanner, and beam-data adapte
|   +-- lattice/                # TRACK lattice elements, conversion helpers, and JSON schem
|   +-- ml/                     # Surrogate models, training, metrics, physics losses, and p
|   +-- sampling/               # Latin-hypercube and Sobol sampling utilities
|   +-- simulation/             # Production dataset generation and TRACK execution orchestr
|   +-- track/                  # TRACK executable wrapper
|   +-- utils/                  # Calibration, RL, TRACK runtime, diagnostics, and parallel
|   +-- visualizations/         # Scanner, phase-space, and plotting utilities
+-- tests/                      # Unit, integration, notebook hygiene, and physics/ML regres
```

Current Maintained Workflows

- `RAON/LEBT_notebooks/` mirrors the maintained LEBT calibration, orbit-correction, and extended surrogate notebooks.
- `RAON/MEBT_notebooks/` contains MEBT lattice creation/loading and segmented TRACK run notebooks. Segmented TRACK runs advance scratch distribution numbers deterministically so each section reads the previous section output.
- `notebooks/` contains the main staged LEBT no-offset Sobol production, validation, MLP training, physics-regularized training, model-comparison, and publication-metric notebooks.
- `models/lebt_no_offset_model_comparison/publication_metrics/` stores publication-oriented diagnostic, component-wise state, derived Courant-Snyder/emittance, and covariance-validity tables.
- Per-run training artifacts use `training_run_artifacts_v2`: `artifact_manifest.json`

records stable paths for checkpoints, metrics, canonical state-physics reports, compatibility reports, plots, summaries, and configuration.
- HDF5 readers detect supported legacy LEBT, no-offset Sobol, and MEBT Sobol layouts through `synapticTrack.data.dataset_compat`; they report unavailable fields explicitly and never fabricate missing state or fixed-input data.
- The GitHub Actions workflow covers `devel`, `main`, and `master`; package metadata includes `PyYAML` so CI installs the YAML dependency used by production configs.
- `docs/site/` is generated from `docs/paper/synapticTrack_consolidated.md` and no longer loads `polyfill.io`.

Maintained LEBT Production Workflow The current production dataset workflow is documented in:

- `docs/lebt_no_offset_sobol_workflow.md`
- `docs/notebook_hygiene.md`
- `docs/TRACK_SETUP.md`
- `docs/mebt_surrogate_input_schema.md`
- `docs/beam_state_physics_api.md`
- `docs/hdf5_dataset_compatibility.md`

Notebook-facing production APIs are maintained through direct workflow submodule imports such as `synapticTrack.simulation.no_offset_sobol_production`. Package initializers retain lazy compatibility exports, but new code should avoid importing complete workflow stacks from `synapticTrack`, `synapticTrack.simulation`, or `synapticTrack.ml`.

Author **Chong Shik Park**
Accelerator Physicist / Machine Learning Researcher
Email: [kuphy@korea.ac.kr]
Affiliation: [Korea University, Sejong]

Status **In development** – contributions, suggestions, and feedback are welcome.

Program Roadmap and Milestones

Source Markdown: `CODEX_TASKS_2026.md`

Codex Milestones

Use one branch or reviewable checkpoint per task.

Task 0 — Repository survey

Read `AGENTS.md` and `notebooks/00_program_specification.ipynb`. Inspect beam classes, backends, scanner analysis, units, packaging, and tests. Do not modify code. Propose a file map and list ambiguities.

Task 1 — Gaussian beam utilities

Implement Twiss-to-covariance conversion and deterministic PyTorch Gaussian sampling. Add tests for covariance, sampled moments, invalid beta/emittance, dtype/device, and reproducibility. Add a notebook that imports the module.

Task 2 — LEBT simulation adapter

Implement backend-neutral request/result schemas and an adapter for the existing LEBT backend. Capture state, controls, diagnostics, last-WS state, exit state, units, seed, validity mask, and failure reason.

Task 3 — Dataset generator

Implement Sobol-sampled, resumable LEBT data generation with validated configuration, deterministic batch seeds, dry-run mode, metadata, and a smoke-test dataset.

Task 4 — MLP surrogate

Implement normalization, multi-output PyTorch MLP training, grouped splits, checkpoints, metrics, plots, and reload tests.

Task 5 — Physics-regularized surrogate

Add positive-definite covariance outputs and moment–Twiss consistency losses. Compare with the MLP using identical data and splits.

Task 6 — Beam-state inference

Implement a permutation-invariant set encoder and bounded differentiable MAP refinement through a frozen surrogate. Test synthetic recovery and identifiability.

Task 7 — MEBT surrogate

Implement MEBT data generation and surrogate modeling with injected beam state, corrector settings, and quadrupole settings. Include correction trajectories.

Task 8 — RL environment and algorithms

Implement a Gymnasium-compatible surrogate environment, normalized target-centered rewards, bounded incremental actions, loss termination, SAC, TD3, response-matrix/SVD baseline, and optimizer baseline.

Task 9 — Single-WS inversion

Implement direct inversion from one WS image/profile plus settings to the Gaussian beam state, followed by optional differentiable refinement.

Task 10 — High-fidelity differentiable tracking

Implement a deterministic external-tracker adapter and compare finite-difference, local-neural, and custom-backward gradient methods.

Task 11 — Generative reconstruction

Implement a conditional VAE baseline, then conditional diffusion with forward-consistency evaluation and uncertainty ensembles.

Source Markdown: `docs/codex_tasks/README_CODEX_TASKS.md`

Codex Markdown Task Documents for LEBT Surrogate Development

This folder contains separate Codex-ready markdown documents derived from the latest discussion about LEBT surrogate modeling.

Documents

1. `01_input_dimension_strategy.md`

   Explains how to reduce or split the input dimension. Defines optics, orbit-response, and full surrogate model variants.

2. `02_output_storage_strategy.md`

   Defines how to store original diagnostic outputs and final beam-state outputs separately so that different models can use different targets.

3. `03_input_parameter_ranges.md`

   Defines recommended ranges for Twiss parameters, emittances, offsets, angles, current, steerers, and electrostatic quadrupoles.

4. `04_training_sample_requirements.md`

   Defines recommended sample counts, dataset phases, learning curves, and validation split strategy.

Recommended repository placement

Copy this folder into:

`synapticTrack/docs/codex_tasks/`

Recommended full structure:

```
synapticTrack/
+-- AGENTS.md
+-- CODEX_TASKS_2026.md
+-- docs/
|   +-- codex_tasks/
|       +-- README_CODEX_TASKS.md
|       +-- 01_input_dimension_strategy.md
|       +-- 02_output_storage_strategy.md
|       +-- 03_input_parameter_ranges.md
|       +-- 04_training_sample_requirements.md
+-- notebooks/
```

Recommended Codex usage

Give Codex one document at a time.

Start with:

```
Read AGENTS.md and docs/codex_tasks/01_input_dimension_strategy.md.
Inspect the existing dataset generation and training pipeline.
Do not modify files yet. Summarize how the current code maps to the requested input-view str
```

Then allow implementation only after reviewing Codex's survey.

Input Parameters, Output Storage, and Sampling Requirements

Source Markdown: `docs/codex_tasks/01_input_dimension_strategy.md`

01 — Input Dimension Strategy for LEBT Surrogate Models

Purpose

This document defines how to handle the LEBT surrogate input dimension after Task 3 dataset generation.

Current situation:

- Current input dimension: approximately 24D.
- Current output: diagnostic quantities, currently 12D.
- The current input includes initial beam Twiss parameters, emittances, initial offsets/angles, beam current, steering magnet strengths, and electrostatic quadrupole strengths.

Main question:

> Should `x0`, `xp0`, `y0`, `yp0`, and `I_beam` be removed from the surrogate input?

Recommendation

Do not force one large 24D model to serve all purposes. Instead, split the work into separate surrogate families:

1. optics surrogate,
2. orbit-response surrogate,
3. optional full absolute diagnostic surrogate.

This reduces sample complexity and makes each model physically interpretable.

---

1. Can `x0`, `xp0`, `y0`, `yp0` be removed?

Yes, but only for models that do not need to predict absolute beam centroids from arbitrary injected offsets.

The variables

$$x_0, \quad x_0', \quad y_0, \quad y_0'$$

mainly control centroid/orbit propagation. For linear optics and central moments, centroid motion and rms-size evolution are approximately separable.

Therefore:

- For rms-size and Twiss propagation, they can often be removed.
- For absolute centroid prediction, they should either be kept or the target should be changed to centroid shift.

---

2. Recommended model split

Model A: Optics surrogate Purpose:

- predict rms beam size,
- predict output covariance/moments,
- predict Twiss/emittance propagation,
- support phase-space reconstruction.

Suggested input:

$$[\alpha_x, \beta_x, \epsilon_x, \alpha_y, \beta_y, \epsilon_y, I_{\text{beam}}, \text{equad}_1, \ldots, \text{equad}_{N_q}]$$

If current is fixed or weakly influential, use:

$$[\alpha_x, \beta_x, \epsilon_x, \alpha_y, \beta_y, \epsilon_y, \text{equad}_1, \ldots, \text{equad}_{N_q}]$$

Do not include:

$$x_0, \quad x_0', \quad y_0, \quad y_0'$$

unless strong nonlinear orbit-size coupling or aperture clipping is important.

Suggested output:

- rms sizes at diagnostic locations,
- final beam moments,
- Twiss/emittance at last wire scanner and LEBT exit.

Model B: Orbit-response surrogate Purpose:

- support LEBT orbit correction,
- avoid dependence on unknown injected centroid/angle,
- support real experimental validation where initial state may be unknown.

Suggested input:

$$[\Delta\text{steerer}_1, \ldots, \Delta\text{steerer}_{N_s}, \text{equad}_1, \ldots, \text{equad}_{N_q}]$$

or

$$[\text{steerer}_1, \ldots, \text{steerer}_{N_s}, \text{equad}_1, \ldots, \text{equad}_{N_q}]$$

Suggested output:

$$[\Delta x_c(s_1), \Delta y_c(s_1), \ldots, \Delta x_c(s_N), \Delta y_c(s_N)]$$

where

$$\Delta x_c(s_i) = x_c(s_i; \mathbf{u}) - x_c(s_i; \mathbf{u}_{\text{ref}})$$

and similarly for y.

This model does not require explicit initial offsets if the reference orbit is measured.

Model C: Full absolute diagnostic surrogate Purpose:

- full simulation replacement,
- absolute prediction of centroid and rms diagnostics,
- useful for synthetic-data studies and model completeness.

Suggested input:

$$[\alpha_x, \beta_x, \epsilon_x, \alpha_y, \beta_y, \epsilon_y, x_0, x_0', y_0, y_0', I_{\text{beam}}, \text{steerers}, \text{equads}]$$

Suggested output:

$$[x_c(s_i), y_c(s_i), \sigma_x(s_i), \sigma_y(s_i)]_{i=1}^{N}$$

This is the most complete model but requires more training samples.

---

3. What to do with beam current?

Beam current I_{beam} should be handled according to physics relevance.

Option 1: fixed-current model Use this if all simulation and experiment are done near one nominal current:

$$I = I_{\text{nom}}$$

Do not include current as input.

Option 2: narrow current model Use this if current varies mildly:

$$I \in [0.8I_{\text{nom}}, 1.2I_{\text{nom}}]$$

Include current as input only in optics/state models.

Option 3: current-indexed models Train separate models:

$$F_\theta^{I_1}, \quad F_\theta^{I_2}, \quad F_\theta^{I_3}$$

This is useful if power-supply or beam-current settings are discrete.

Option 4: full current-dependent model Use if current strongly affects space charge and rms evolution:

$$I \in [0.5I_{\text{nom}}, 1.5I_{\text{nom}}]$$

This requires more data.

---

4. Recommended implementation decision

For the next Codex milestone, implement model-selection support in the dataset loader.

Do not delete existing input columns. Instead, allow different input views:

```
input_view = "full"
input_view = "optics"
input_view = "orbit_response"
```

Suggested behavior:

- `full`: all available inputs.
- `optics`: Twiss, emittance, optional current, equads.
- `orbit_response`: steerers, equads, possibly delta steerers.
- `custom`: user-provided list of input columns.

---

5. Suggested repository changes

Possible files:

```
src/synaptictrack/data/input_views.py
src/synaptictrack/data/schemas.py
src/synaptictrack/data/datasets.py
tests/test_input_views.py
```

Suggested functionality:

```
get_input_columns(view: str, *, include_current: bool = True) -> list[str]
select_input_view(dataframe, view: str, include_current: bool = True)
```

---

6. Required tests

Codex should add tests that verify:

1. the original full input view still works;
2. the optics input view removes `x0`, `xp0`, `y0`, `yp0`;
3. the optics input view optionally includes or excludes `I_beam`;

4. the orbit-response view includes steering and equad variables;
5. input column ordering is deterministic;
6. invalid view names raise a clear error;
7. training code can accept different input dimensions.

---

7. Codex prompt

Use the following prompt for Codex:

```
Read AGENTS.md, CODEX_TASKS_2026.md, and this document:
docs/codex_tasks/01_input_dimension_strategy.md.

Task:
Implement input-view support for LEBT surrogate datasets.

Requirements:
1. Do not remove existing dataset columns.
2. Add reusable input-view utilities.
3. Support at least:
   - full
   - optics
   - orbit_response
   - custom
4. In the optics view, allow `include_current=True/False`.
5. Preserve deterministic column ordering.
6. Add tests for all input views.
7. Update the relevant notebook to demonstrate training with the reduced optics input.
8. Do not modify the simulation backend.
9. Do not implement MEBT or RL in this task.

After implementation, summarize changed files, tests run, and remaining assumptions.
```

Source Markdown: `docs/codex_tasks/02_output_storage_strategy.md`

02 — Output Storage Strategy for Diagnostics and Beam-State Targets

Purpose

This document defines how to store original diagnostic outputs and additional beam-state outputs so that different surrogate models can use different targets.

Main question:

> Can original diagnostic outputs and beam-state outputs be stored separately and reused by different models?

Answer:

Yes. This is strongly recommended.

---

1. Core recommendation

Do not store all targets only as one flat array.

Instead, store outputs in separate named groups:

```
output/
+-- diagnostics/
+-- orbit_response/
+-- state_last_ws/
+-- state_lebt_exit/
```

This allows the same simulation dataset to support multiple models:

- diagnostic-only surrogate,
- orbit-response surrogate,
- final-state surrogate,
- multi-task surrogate,
- phase-space inference model.

---

2. Recommended output groups

2.1 `y_diag` Original diagnostic outputs.

Possible contents:

$$[x_c(s_1), y_c(s_1), \sigma_x(s_1), \sigma_y(s_1), \ldots, x_c(s_N), y_c(s_N), \sigma_x(s_N), \sigma_y(s_N)]$$

If the current project output is 12D, keep it exactly as-is for backward compatibility.

Do not break the existing training pipeline.

2.2 `y_orbit_response` Centroid changes relative to a reference setting.

$$\Delta x_c(s_i) = x_c(s_i; \mathbf{u}) - x_c(s_i; \mathbf{u}_{\text{ref}})$$

$$\Delta y_c(s_i) = y_c(s_i; \mathbf{u}) - y_c(s_i; \mathbf{u}_{\text{ref}})$$

Suggested array:

$$[\Delta x_c(s_1), \Delta y_c(s_1), \ldots, \Delta x_c(s_N), \Delta y_c(s_N)]$$

This target is useful for orbit correction because it avoids requiring exact knowledge of the initial injected centroid.

2.3 `y_state_last_ws` Beam state at the last wire scanner.

Recommended contents:

$$[x_c,\ x'_c,\ y_c,\ y'_c, \langle x^2\rangle, \langle xx'\rangle, \langle x'^2\rangle, \langle y^2\rangle, \langle yy'\rangle, \langle y'^2\rangle]$$

This is a 10D state target.

Also store derived Twiss values as either metadata or optional target:

$$[\alpha_x, \beta_x, \epsilon_x, \alpha_y, \beta_y, \epsilon_y]$$

2.4 `y_state_lebt_exit` Beam state at the LEBT exit.

Use the same 10D structure:

$$[x_c,\ x'_c,\ y_c,\ y'_c, \langle x^2\rangle, \langle xx'\rangle, \langle x'^2\rangle, \langle y^2\rangle, \langle yy'\rangle, \langle y'^2\rangle]$$

This is the most important output for the later MEBT surrogate because it becomes the injected beam state.

---

3. Why moments are preferred over Twiss as training targets

The central second moments are

$$\langle x^2\rangle, \quad \langle xx'\rangle, \quad \langle x'^2\rangle.$$

From these,

$$\epsilon_x = \sqrt{\langle x^2\rangle\langle x'^2\rangle - \langle xx'\rangle^2}$$

$$\beta_x = \frac{\langle x^2\rangle}{\epsilon_x}$$

$$\alpha_x = -\frac{\langle xx' \rangle}{\epsilon_x}.$$

Moments are closer to raw particle statistics and are generally more stable as supervised-learning labels.

Therefore:

- train primarily on moments;
- derive Twiss for analysis;
- optionally include Twiss in metadata or auxiliary targets.

---

4. Dataset schema recommendation

A sample should support this structure:

```
sample/
+-- input/
|   +-- twiss
|   +-- emittance
|   +-- initial_centroid
|   +-- current
|   +-- steerers
|   +-- equads
+-- output/
|   +-- diagnostics
|   +-- orbit_response
|   +-- state_last_ws
|   +-- state_lebt_exit
+-- metadata/
|   +-- units
|   +-- seed
|   +-- lattice_version
|   +-- tracking_code
|   +-- valid_flag
|   +-- failure_reason
```

If using HDF5 or Zarr, use groups. If using tabular formats, use column prefixes such as:

```
diag/x_c_WS01
diag/y_c_WS01
diag/sigma_x_WS01
diag/sigma_y_WS01

state_exit/x_c
```

```
state_exit/xp_c
state_exit/y_c
state_exit/yp_c
state_exit/x2
state_exit/xxp
state_exit/xp2
state_exit/y2
state_exit/yyp
state_exit/yp2
```

---

5. Target selection API

The dataset loader should support different target views:

```
target_view = "diagnostics"
target_view = "orbit_response"
target_view = "state_exit"
target_view = "state_last_ws"
target_view = "diag_plus_exit_state"
target_view = "diag_plus_all_states"
target_view = "custom"
```

Examples:

```
dataset = LEBTSurrogateDataset(
    path="data/lebt_dataset_v3.h5",
    input_view="optics",
    target_view="state_exit",
)
```

```
dataset = LEBTSurrogateDataset(
    path="data/lebt_dataset_v3.h5",
    input_view="full",
    target_view="diag_plus_all_states",
)
```

---

6. Multi-task output strategy

A multi-task surrogate can predict:

$$\mathbf{y} = [\mathbf{y}_{\text{diag}}, \mathbf{y}_{\text{state,lastWS}}, \mathbf{y}_{\text{state,exit}}]$$

The training loss should be grouped:

$$\mathcal{L} = \lambda_{\mathrm{diag}}\mathcal{L}_{\mathrm{diag}} + \lambda_{\mathrm{lastWS}}\mathcal{L}_{\mathrm{lastWS}} + \lambda_{\mathrm{exit}}\mathcal{L}_{\mathrm{exit}}.$$

Do not force all outputs to have equal weight. Normalize each group separately.

---

7. Required tests

Codex should add tests that verify:

1. original diagnostic output path remains unchanged;
2. `y_diag` can be loaded independently;
3. `y_state_last_ws` can be loaded independently;
4. `y_state_lebt_exit` can be loaded independently;
5. combined target views concatenate in deterministic order;
6. derived Twiss parameters from moments are physically valid;
7. covariance determinants are positive for generated labels;
8. old notebooks still run with diagnostic-only output.

---

8. Codex prompt

Use the following prompt for Codex:

```
Read AGENTS.md, CODEX_TASKS_2026.md, and this document:
docs/codex_tasks/02_output_storage_strategy.md.

Task:
Add separate output-target storage and target-view loading for LEBT surrogate datasets.

Requirements:
1. Preserve the existing diagnostic output exactly.
2. Add separate groups or column prefixes for:
   - diagnostics
   - orbit_response
   - state_last_ws
   - state_lebt_exit
3. For beam-state outputs, store central moments and centroids/angles.
4. Store derived Twiss/emittance as metadata or optional targets.
5. Add target-view selection in the dataset loader.
6. Support combined target views for multi-task learning.
7. Add tests for shape, ordering, backward compatibility, and physical validity.
8. Update the notebook to demonstrate diagnostic-only and diagnostic+state training.
9. Do not implement RL or MEBT in this task.

After implementation, summarize changed files, tests run, and remaining assumptions.
```

Source Markdown: `docs/codex_tasks/03_input_parameter_ranges.md`

03 — Recommended Input Parameter Ranges for LEBT Surrogate Data Generation

Purpose

This document defines recommended sampling ranges for LEBT surrogate model data generation.

Main question:

> What ranges should be used for input parameters?

The exact values should ultimately come from the RAON LEBT nominal optics, hardware limits, and trusted simulation acceptance. Until those are fully fixed, use relative ranges around nominal values.

---

1. Range strategy

Use three nested ranges:

1. nominal range,
2. training range,
3. stress-test range.

Do not use one wide uniform box for everything.

The recommended strategy is:

- train densely near normal operation;
- include enough mismatch for robustness;
- reserve extreme cases for validation and stress testing;
- avoid wasting samples in always-lost or physically irrelevant regions.

---

2. Twiss parameter ranges

Let the nominal input Twiss parameters be:

$$\alpha_{x0}, \quad \beta_{x0}, \quad \epsilon_{x0}$$

and similarly for the y plane.

2.1 Training range Use:

$$\alpha_x \in [\alpha_{x0} - 2, \alpha_{x0} + 2]$$

$$\beta_x \in [0.5\beta_{x0}, 2.0\beta_{x0}]$$

$$\epsilon_x \in [0.5\epsilon_{x0}, 2.0\epsilon_{x0}]$$

Apply the same structure to y:

$$\alpha_y \in [\alpha_{y0} - 2, \alpha_{y0} + 2]$$

$$\beta_y \in [0.5\beta_{y0}, 2.0\beta_{y0}]$$

$$\epsilon_y \in [0.5\epsilon_{y0}, 2.0\epsilon_{y0}]$$

2.2 Stress-test range Use:

$$\alpha_x \in [\alpha_{x0} - 4, \alpha_{x0} + 4]$$

$$\beta_x \in [0.25\beta_{x0}, 4.0\beta_{x0}]$$

$$\epsilon_x \in [0.25\epsilon_{x0}, 3.0\epsilon_{x0}]$$

Again, apply the same structure to y.

Stress-test samples should be used primarily for validation, not as the dominant training population.

---

3. Initial offsets and angles

If included in the full surrogate:

3.1 Nominal training range

$$x_0, y_0 \in [-2, 2] \text{ mm}$$

$$x_0', y_0' \in [-10, 10] \text{ mrad}$$

3.2 Stress-test range

$$x_0, y_0 \in [-5, 5] \text{ mm}$$

$$x_0', y_0' \in [-30, 30] \text{ mrad}$$

If these values exceed the realistic LEBT acceptance, reduce them. Do not oversample regions where the beam is always lost.

---

4. Beam current

If current is fixed, do not include it as an input.

If included, use one of the following.

4.1 Narrow current model

$$I \in [0.8 I_{\text{nom}}, 1.2 I_{\text{nom}}]$$

Recommended for the first current-dependent surrogate.

4.2 General current model

$$I \in [0.5 I_{\text{nom}}, 1.5 I_{\text{nom}}]$$

Use only after the narrow model works.

4.3 Stress-test current range

$$I \in [0, 2 I_{\text{nom}}]$$

Use for validation, not first training.

---

5. Steering magnet ranges

If simulation uses kick angles:

$$\theta_i \in [-\theta_{i,\max}, \theta_{i,\max}]$$

However, the training set should be denser in the operational region:

$$\theta_i \in [-0.5\theta_{i,\max}, 0.5\theta_{i,\max}]$$

For real experimental validation, prefer calibrated hardware variables:

$$V_i \in [V_{i,\min}, V_{i,\max}]$$

or

$$I_i \in [I_{i,\min}, I_{i,\max}]$$

with conversion to kick angle handled by an actuator model.

---

6. Electrostatic quadrupole ranges

Let G_{j0} be the nominal equad strength.

6.1 First training range

$$G_j \in [0.8G_{j0}, 1.2G_{j0}]$$

6.2 Wider training range

$$G_j \in [0.6G_{j0}, 1.4G_{j0}]$$

6.3 Hardware-limit range

$$G_j \in [G_{j,\min}, G_{j,\max}]$$

Use the hardware-limit range only when needed for recovery or robustness studies.

---

7. Recommended dataset versions

Dataset v1: optics surrogate Purpose:

- rms and beam-state propagation.

Input:

$$[\alpha_x, \beta_x, \epsilon_x, \alpha_y, \beta_y, \epsilon_y, \text{equads}]$$

Optional:

$$I_{\text{beam}}$$

Ranges:

$$\alpha = \alpha_0 \pm 2$$

$$\beta = 0.5\text{–}2.0 \times \beta_0$$

$$\epsilon = 0.5\text{–}2.0 \times \epsilon_0$$

$$G = 0.8\text{–}1.2 \times G_0$$

Dataset v2: orbit-response surrogate Purpose:

- LEBT orbit correction.

Input:

$$[\text{steerers}, \text{equads}]$$

Output:

$$[\Delta x_c, \Delta y_c]$$

Ranges:

$$\theta = 0.5\text{–}1.0 \times \text{operational steering range}$$

$$G = 0.8\text{–}1.2 \times G_0$$

Dataset v3: full surrogate Purpose:

- complete absolute diagnostic prediction.

Input:

$$[\alpha, \beta, \epsilon, x_0, x_0', y_0, y_0', I, \text{steerers}, \text{equads}]$$

Use after v1 and v2 are validated.

---

8. Sampling method

Use Sobol or Latin-hypercube sampling.

Recommended mixture:

- 60% nominal/training range,
- 25% mismatch/recovery range,
- 15% stress-test or boundary range.

For each simulation sample, store:

- input vector,
- physical parameter values before normalization,
- validity flag,
- failure reason,
- aperture/loss status,
- random seed,
- lattice version,
- units.

---

9. Required tests

Codex should add tests that verify:

1. generated ranges obey configuration bounds;
2. beta and emittance are always positive;
3. current is non-negative;
4. steering and equad settings obey bounds;
5. stress-test ranges are separated from training ranges;
6. invalid range configurations raise clear errors;
7. sampling is reproducible for a fixed seed;
8. generated samples can be passed to the simulation adapter.

---

10. Codex prompt

Use the following prompt for Codex:

```
Read AGENTS.md, CODEX_TASKS_2026.md, and this document:
docs/codex_tasks/03_input_parameter_ranges.md.

Task:
Implement configurable input-parameter range definitions for LEBT surrogate
data generation.

Requirements:
1. Define range configuration objects for:
```

```
   - Twiss parameters
   - emittances
   - initial offsets and angles
   - beam current
   - steering magnets
   - electrostatic quadrupoles
2. Support nominal, training, and stress-test ranges.
3. Support dataset presets:
   - optics
   - orbit_response
   - full
4. Use Sobol or Latin-hypercube sampling if already available; otherwise
   implement a clean interface and use a deterministic fallback.
5. Validate all physical bounds.
6. Add tests for reproducibility, bounds, invalid configs, and dataset presets.
7. Do not modify the tracking backend.
8. Do not train neural networks in this task.

After implementation, summarize changed files, tests run, and remaining assumptions.
```

Source Markdown: `docs/codex_tasks/04_training_sample_requirements.md`

04 — Training Sample Requirements for LEBT Surrogate Models

Purpose

This document estimates how much simulation data is needed for training the LEBT surrogate models.

Main question:

> How many samples are needed for training the surrogate model?

The answer depends on:

- input dimension,
- output dimension,
- nonlinearities,
- model type,
- sampling range,
- whether absolute outputs or response shifts are predicted,
- simulation noise from macroparticle statistics.

---

1. Summary recommendation

Do not generate a huge full 24D production dataset immediately.

Use a staged data-generation strategy:

1. smoke-test dataset,
2. first real dataset,
3. production reduced-input dataset,
4. production full-input dataset,
5. active-learning refinement.

---

2. Recommended sample counts

2.1 Smoke-test dataset Purpose:

- verify code,
- verify dataset schema,
- verify normalization,
- verify training loop,
- verify notebook execution.

Recommended size:

$$N = 500\text{–}2{,}000$$

Do not use this for scientific conclusions.

2.2 First real model Purpose:

- determine whether the input/output definition is reasonable,
- compare MLP and physics-regularized MLP,
- inspect error distributions.

Recommended size:

$$N = 10{,}000\text{–}30{,}000$$

This is suitable for first useful model comparisons.

2.3 Reduced-input optics surrogate Purpose:

- rms-size and beam-state propagation,
- Twiss/emittance dependence,
- phase-space reconstruction support.

Approximate input dimension:

$$6 + N_{\text{equad}}$$

or

$$7 + N_{\text{equad}}$$

if current is included.

Recommended size:

$$N = 50{,}000\text{–}100{,}000$$

2.4 Orbit-response surrogate Purpose:

- orbit correction,
- steering response,
- experimental RL pretraining.

Approximate input dimension:

$$N_{\text{steerer}} + N_{\text{equad}}$$

Recommended size:

$$N = 10{,}000\text{–}50{,}000$$

If the response is close to linear, the lower end may be sufficient.

2.5 Full 24D surrogate Purpose:

- absolute diagnostic prediction over a broad operating range.

Recommended size:

$$N = 100{,}000\text{–}300{,}000$$

If the model includes wide current variation, large offsets, strong nonlinearities, and beam-state outputs, use more data.

2.6 Full model with diagnostic and state outputs Purpose:

- publication-quality full surrogate,
- diagnostic prediction,
- final-state prediction,
- uncertainty analysis.

Recommended size:

$$N = 200{,}000\text{–}500{,}000$$

Use this only after reduced models are validated.

---

3. Why the full 24D model needs many samples

A 24D input space is large. Uniformly covering it is inefficient.

The required sample size increases when:

- many inputs are weakly relevant but included anyway,
- output includes both centroids and beam-state moments,
- current-dependent dynamics are included,
- large offset/angle ranges cause nonlinear or aperture effects,
- the training region includes both nominal and stress-test conditions.

Therefore, reducing the input dimension by model purpose is strongly recommended.

---

4. Active-learning strategy

Use iterative data generation instead of one large blind dataset.

Recommended loop:

1. generate $N = 5{,}000$ samples;
2. train a quick MLP;
3. evaluate error versus input parameters;
4. identify high-error regions;
5. add $N = 20{,}000$ Sobol samples;
6. retrain;
7. add focused samples near high-error or high-gradient regions;
8. repeat until validation error saturates.

This is more efficient than generating hundreds of thousands of samples immediately.

---

5. Learning-curve requirement

Codex should implement tools to generate learning curves.

Train the same model with:

$$N = [1{,}000, 3{,}000, 10{,}000, 30{,}000, 100{,}000]$$

when available.

Plot:

$$\text{MAE}(N), \qquad \text{RMSE}(N), \qquad R^2(N), \qquad \text{physics violation}(N).$$

This determines whether more data are still useful.

---

6. Validation split strategy

Do not use only random splits.

Use:

1. random interpolation split;
2. held-out Twiss/emittance region;
3. held-out equad range;
4. held-out steering range;
5. stress-test region;
6. closed-loop correction trajectory validation.

For each split, report separate metrics.

---

7. Suggested dataset phases

Phase A: smoke dataset

$$N = 1{,}000$$

Use in CI tests and notebooks.

Phase B: pilot dataset

$$N = 10{,}000\text{–}30{,}000$$

Use for first MLP and physics-regularized comparison.

Phase C: reduced production dataset

$$N = 50{,}000\text{–}100{,}000$$

Use for optics and orbit-response models.

Phase D: full production dataset

$$N = 100{,}000\text{–}300{,}000$$

Use only after the target schema and model split are stable.

Phase E: active-learning refinement Add focused samples in high-error regions.

---

8. Required tests and utilities

Codex should implement:

1. configurable dataset-size presets;
2. smoke-test dataset generator;
3. learning-curve training script;
4. grouped validation split utilities;
5. error-by-region reporting;
6. reproducibility check for fixed seed;
7. runtime metadata logging.

---

9. Codex prompt

Use the following prompt for Codex:

```
Read AGENTS.md, CODEX_TASKS_2026.md, and this document:
docs/codex_tasks/04_training_sample_requirements.md.

Task:
Add dataset-size presets, learning-curve utilities, and validation split
support for LEBT surrogate model development.

Requirements:
1. Support dataset-size presets:
   - smoke: 500-2,000 samples
   - pilot: 10,000-30,000 samples
   - reduced_production: 50,000-100,000 samples
   - full_production: 100,000-300,000 samples
```

```
2. Add grouped validation split utilities:
   - random
   - held-out Twiss/emittance region
   - held-out equad range
   - held-out steering range
   - stress-test region
3. Add a learning-curve configuration that can train on increasing subsets.
4. Add metrics grouped by output target and validation region.
5. Ensure the smoke preset is small enough for CI or quick notebook execution.
6. Do not generate a large dataset by default.
7. Do not implement MEBT or RL in this task.

After implementation, summarize changed files, tests run, and remaining assumptions.
```

Physics Models and Machine Conventions

Source Markdown: `docs/LEBT_STEERING_CURRENT_KICK_CONVERSION.md`

LEBT Steering Current to Kick Conversion

Current Internal Convention

The existing synapticTrack LEBT simulation, no-offset Sobol dataset, validation workflow, and MLP training pipeline use steering magnet kick angles in `mrad`.

The existing 19D sampled input remains:

```
alpha_x, beta_x, emittance_x,
alpha_y, beta_y, emittance_y,
SM0_H, SM1_H, SM2_H, SM3_H, SM4_H,
SM0_V, SM1_V, SM2_V, SM3_V, SM4_V,
alpha_0, alpha_1, alpha_2
```

The steering entries `SM0_H ... SM4_V` are kick values in `mrad`.

Machine Current Convention

The real RAON LEBT steering magnets are controlled by power-supply current in amperes. The current assumed linear calibration is:

```
1 mrad kick = 0.168 A
```

The nominal power-supply current step is:

```
0.01 A
```

Therefore:

```
kick_mrad = current_A / 0.168
current_A = 0.168 * kick_mrad
0.01 A = 0.0595238 mrad
```

Conversion Equations

The implemented sign and offset convention is:

```
kick_mrad = polarity * (current_A - current_offset_A) / amp_per_mrad
current_A = current_offset_A + polarity * amp_per_mrad * kick_mrad
```

The default calibration is:

```
amp_per_mrad = 0.168
current_step_A = 0.01
current_offset_A = 0.0
polarity = +1
```

Each steering magnet can override:

```
amp_per_mrad
current_offset_A
current_min_A
current_max_A
current_step_A
polarity
```

The optional config file is:

```
configs/hardware/lebt_steering_calibration.yaml
```

If that file is missing, built-in defaults are used.

Quantization

Current quantization is:

```
I_quantized = round(I / current_step_A) * current_step_A
```

Quantized kick is computed by converting the quantized current:

```
kick_quantized_mrad = I_quantized / amp_per_mrad
```

With the default calibration:

```
kick_step_mrad = 0.01 / 0.168 = 0.0595238 mrad
```

Quantization is opt-in. Simulation-only dataset generation and MLP training should normally leave quantization disabled.

Policy Options

Option A: Simulation-Native Training and simulation use steering kick in `mrad`. Machine current is converted only at the hardware boundary.

This is the current default and the recommended policy for the existing no-offset Sobol dataset and MLP surrogate training.

Option B: Hardware-Native ML or RL actions use power-supply current in `A`. Before simulation or surrogate evaluation, currents are converted to kicks in `mrad`.

This is recommended for future real-machine RL deployment because the action variable matches the physical actuator.

Option C: Dual Representation Datasets may explicitly store both:

```
steering_kick_mrad
steering_current_A
```

Training can then choose a representation through an explicit opt-in input view. This is useful for future experiment/simulation alignment, but it is not enabled by default.

Current Training Recommendation

Keep the current production dataset and MLP training in kick units. The simulation naturally accepts mrad kicks, and the existing 19D Sobol dataset is already defined that way.

Use:

```
from synapticTrack.hardware import SteeringActuatorAdapter

adapter = SteeringActuatorAdapter()
kick_dict = adapter.currents_to_kicks(current_dict, quantize=True)
```

only when translating machine-facing current settings to simulation-facing kicks.

Future Real-Machine RL Recommendation

Use current in `A` as the RL action variable at the hardware boundary. Convert to `mrad` before simulation or surrogate evaluation:

```
kick_dict = adapter.currents_to_kicks(current_dict, quantize=True)
```

This keeps the policy action space aligned with the real power supplies while preserving synapticTrack's internal mrad convention.

Limitations

The current implementation assumes a linear current-to-kick calibration. Future machine studies should add:

- beam-based calibration per magnet;
- hysteresis characterization;
- polarity verification;
- current limits from hardware;
- reproducibility checks after cycling magnets;
- time-dependent or nonlinear correction terms if needed.

Those effects are deliberately not assumed in the current layer.

Source Markdown: `docs/lebt_production_dataset_configs.md`

LEBT Production Dataset Configurations

This document records the first versioned production dataset-generation configurations for the Task-3 LEBT surrogate workflow. These configs are intentionally staged and do not launch full production generation by default.

Current Status Relative to Sobol Workflow

These early Task-3 production configs remain useful as design records and for legacy/generic LEBT dataset generation. The maintained no-offset production workflow is now `lebt_production_no_offset_multitarget_sobol_v1`, documented in `docs/lebt_no_offset_sobol_workflow.md`, with a 19D sampled input and 44D target output.

Pilot Audit Decision

The latest pilot audit did not find a pilot-sized current-schema dataset in the workspace. The only current-schema TRACK dataset had 8 samples, and the larger available files were legacy orbit-correction datasets without schema metadata, validity masks, units, failed-sample reasons, or beam-state labels.

Decision for v1 production configs:

- keep the Task-3 `training` ranges instead of widening ranges;
- split production datasets by model type;
- require a 100-sample preflight for each config before full generation;
- include metadata that records the pilot-audit blocker.

Versioned Configs

Config ID	Purpose	Samples	Preset	State Outputs	Default Output
`lebt_production_diag_v1`	full-diagnostic target	50,000	`full`	no	`data/production/lebt_production_di`
`lebt_production_optics_state_v1`	optics-state target	50,000	`optics`	yes	`data/production/lebt_production_op`
`lebt_production_orbit_response_v1`	steering-orbit response	20,000	`orbit_response`	no	`data/production/lebt_production_or`

Config files live in `configs/lebt/`.

Metadata Saved

The config loader injects these fields into dataset metadata:

- `production_config_id`
- `production_config_version`
- `git_commit_hash`
- `lattice_version`
- `backend_name`
- `units`
- `random_seed`
- `sampling_method`
- `parameter_ranges`
- `generation_timestamp`
- `pilot_audit_status`

The full resolved `LEBTDatasetConfig` is also saved in generated HDF5 `config_json` by the dataset generator. Input-only dry runs save the same resolved config metadata.

Dry-Run Input Tables

Dry-run mode for production configs writes only sampled inputs and summary statistics. It does not create target arrays and does not call TRACK.

```
python3 -m synapticTrack.cli lebt-production-dry-run lebt_production_diag_v1
python3 -m synapticTrack.cli lebt-production-dry-run lebt_production_optics_state_v1
python3 -m synapticTrack.cli lebt-production-dry-run lebt_production_orbit_response_v1
```

Optional overrides:

```
python3 -m synapticTrack.cli lebt-production-dry-run lebt_production_diag_v1 \
  --n-samples 1000 \
  --output-file data/dry_run/lebt_production_diag_v1_inputs_1000.h5 \
  --summary-file data/dry_run/lebt_production_diag_v1_inputs_1000.csv
```

Preflight Commands

These commands print a 100-sample preflight command by default:

```
python3 -m synapticTrack.cli lebt-production-preflight lebt_production_diag_v1
python3 -m synapticTrack.cli lebt-production-preflight lebt_production_optics_state_v1
python3 -m synapticTrack.cli lebt-production-preflight lebt_production_orbit_response_v1
```

To actually run the preflight after reviewing dry-run inputs and TRACK availability, add `--execute`:

```
python3 -m synapticTrack.cli lebt-production-preflight lebt_production_diag_v1 --execute
python3 -m synapticTrack.cli lebt-production-preflight lebt_production_optics_state_v1 --exe
python3 -m synapticTrack.cli lebt-production-preflight lebt_production_orbit_response_v1 --e
```

Do not run full production generation until each preflight has been audited for validity fraction, failed/lost rows, output ranges, units, and beam-state moment validity where applicable.

Full Production Commands

Full generation is intentionally not wrapped as a one-line default command in this task. After successful preflight review, run production through the config loader or a reviewed operations script that calls `generate_lebt_dataset(config.to_lebt_config(dry_run=False))` for the selected config.

Simulation Code Structure and External Codes

Source Markdown: `docs/TRACK_SETUP.md`

TRACK Setup

`synapticTrack` does not commit or vendor the external TRACK executable. The repository-local convention is:

```
synapticTrack/
+-- TRACK/
|   +-- TRACKv39C.exe
+-- bin/
|   +-- helper commands
```

`TRACK/` is local-only and ignored by Git. Create it yourself:

```
mkdir -p TRACK
## place TRACKv39C.exe and any local runtime files under TRACK/
chmod +x TRACK/TRACKv39C.exe
```

The default resolved executable is:

`TRACK/TRACKv39C.exe`

Override it with an environment variable:

```
export SYNAPTICTRACK_TRACK_EXE=/custom/path/to/TRACKv39C.exe
bin/syntrack-track-check
```

Or use a config override:

```
track_exe_path: /custom/path/to/TRACKv39C.exe
```

Resolution precedence is:

1. explicit config or command-line path;
2. `SYNAPTICTRACK_TRACK_EXE`;
3. repository-local `TRACK/TRACKv39C.exe`.

Helper Commands

Check TRACK path:

```
bin/syntrack-track-check
bin/syntrack-track-check --track-exe TRACK/TRACKv39C.exe
```

Inspect preflight settings without running production:

```
bin/syntrack-lebt-preflight
```

Run only the 240-sample preflight:

```
bin/syntrack-lebt-preflight --run
```

Validate an existing no-offset Sobol dataset:

```
bin/syntrack-lebt-validate
bin/syntrack-lebt-validate data/processed/lebt_production_no_offset_multitarget_sobol_v1 --w
```

Installed console scripts expose the same commands when the package is installed:

```
syntrack-track-check
syntrack-lebt-preflight
syntrack-lebt-validate
```

Segmented TRACK Runs

RAON MEBT notebooks use segmented TRACK execution. The initial beam distribution number must match the source lattice convention, and each segment must write the next scratch distribution number for the following segment. For example, an MEBT run starting from `scratch.#07` writes `scratch.#08` after the first segment, then the next segment reads `scratch.#08`. This preserves full-lattice ordering while exposing intermediate diagnostic outputs.

Production Notebooks

Production notebooks display the resolved TRACK path and whether it exists before generation. They should not contain machine-specific TRACK paths in committed source or outputs.

Beam Current Unit Contract

The repository does not contain an authoritative TRACK manual statement for the `current` namelist unit. `synapticTrack` therefore uses `uA` as its canonical beam-current unit and exposes `track_current_per_synaptic_current` as an explicit conversion boundary. Its current default is `1.0`, solely to preserve existing `track.dat` values such as `current = 0.05`; it is marked `unverified_local_repository` and must not be interpreted as a verified physical unit equivalence. Generated LEBT/MEBT metadata records the canonical value, the TRACK input value, the conversion factor, and the verification status.

Source Markdown: `docs/refactor_baseline_r0.md`

R0 Baseline Characterization: Production Dataset Pipeline

Date: 2026-07-15

Branch inspected: `devel`

Baseline note: this report was created after the local R1 and R2 commits had already been applied:

- `e06fd2e` — R1 schema centralization
- `66c720e` — R2 production config policy and validation

The behavioral baseline below records the current branch state at report time. No production generation, neural-network training, or tracking backend execution was performed for this report.

Scope Inspected

Required files inspected:

- `src/synapticTrack/simulation/no_offset_sobol_production.py`
- `src/synapticTrack/simulation/dataset_generator.py`
- `src/synapticTrack/data/input_views.py`
- `src/synapticTrack/data/target_views.py`
- `src/synapticTrack/simulation/no_offset_sobol_validation.py`
- `notebooks/04_lebt_no_offset_sobol_dataset_production.ipynb`
- `notebooks/05_lebt_no_offset_sobol_dataset_validation.ipynb`

Task documents inspected:

- `AGENTS.md`

- CODEX_TASKS_2026.md
- docs/codex_tasks/01_input_dimension_strategy.md
- docs/codex_tasks/02_output_storage_strategy.md
- docs/codex_tasks/03_input_parameter_ranges.md
- docs/codex_tasks/04_training_sample_requirements.md

Current Expected Behavior

The no-offset Sobol LEBT production workflow currently targets:

- Dataset name: lebt_production_no_offset_multitarget_sobol_v1
- Sampled input dimension: 19D
- Fixed-zero parameters: 5
  - x0
  - xp0
  - y0
  - yp0
  - beam_current
- Effective full simulation parameter dimension: 24D
- Target groups:
  - diagnostics: 24D
  - state_last_ws: 10D
  - state_lebt_exit: 10D
- Total target dimension: 44D
- Target view for multi-task production: diag_plus_all_states
- Input view for no-offset production: full_19d_fixed_injection
- Sobol continuation stages:
  - 200,000 samples
  - 300,000 samples
  - 500,000 samples
- Batch-size default for production config: 2,400
- Preflight config: 240 samples, batch size 120

Sobol continuation is based on global sample indices. Existing completed indices are discovered from batch files, duplicates are rejected, and missing contiguous index ranges are planned as new batches.

Current Modules And Responsibilities

no_offset_sobol_production.py Primary responsibility: specialized no-offset Sobol LEBT production orchestration.

Current responsibilities include:

- SobolConfig and NoOffsetSobolProductionConfig
- production/preflight config loading and validation
- Sobol unit-cube sampling by global index
- physical input mapping

- 19D to full 24D expansion with fixed-zero injection parameters
- duplicate global-index detection
- batch planning and resume/extension behavior
- preflight configuration conversion
- batch execution through `SimulationBackend`
- per-sample work-dir policy and progress printing
- HDF5 batch writing
- config, metadata, generation log, Sobol index, audit, and validity-summary writing
- runtime estimation from preflight output

The module is still broad: about 988 lines at inspection time.

`dataset_generator.py` Primary responsibility: general LEBT dataset generation path used by smoke/pilot/test workflows.

Current responsibilities include:

- LEBT parameter range objects
- dataset presets and range profile resolution
- beamline-neutral `DatasetGenerationConfig`
- LEBT-specific `LEBTDatasetConfig`
- dry-run sampling validation and summary tables
- deterministic seed utilities
- Sobol/LHS input generation
- generic calibrated LEBT dataset generation
- input view materialization
- target group storage for diagnostics, orbit response, and state outputs
- beam-state moment/Twiss conversion and validation
- dataset audit support

The module is about 1,473 lines and overlaps with the production generator in config validation, worker execution, deterministic sampling, HDF5 writing, progress handling, cleanup policy, masks, failure reasons, and metadata.

`input_views.py` Primary responsibility: deterministic input-column selection.

Public behavior:

- `full`: all available input columns
- `optics`: Twiss, emittance, optional current, and equads
- `orbit_response`: steerers plus equads
- `custom`: user-supplied ordered column list

The helper validates missing/duplicated columns and preserves deterministic ordering.

`target_views.py` Primary responsibility: target-view loading from HDF5 datasets.

Public behavior:

- `diagnostics`
- `orbit_response`
- `state_last_ws`
- `state_lebt_exit`
- `diag_plus_exit_state`
- `diag_plus_all_states`
- `custom`

It supports current grouped output paths and legacy flat datasets such as `y_diag`, `y_state_last_ws`, and `y_state_exit`.

`no_offset_sobol_validation.py` Primary responsibility: validate no-offset Sobol production datasets and write validation reports.

Current responsibilities include:

- required-file checks
- metadata/config loading
- Sobol index uniqueness and continuity checks
- batch integrity checks
- fixed-zero parameter checks
- output dimension checks
- diagnostic finite/RMS positivity checks
- moment-to-Twiss conversion
- beam-state physical validity checks
- validation report JSON/Markdown writing

Public Functions And Classes In Inspected Modules

No-offset Sobol production

- `SobolConfig`
- `NoOffsetSobolProductionConfig`
- `replace_config`
- `load_no_offset_sobol_config`
- `load_no_offset_sobol_production_config`
- `load_no_offset_sobol_preflight_config`
- `no_offset_sobol_config_from_mapping`
- `no_offset_sobol_config_to_dict`
- `sampled_bounds`
- `sobol_unit_samples_for_indices`
- `physical_samples_for_indices`
- `expand_19d_to_full_24d`
- `duplicate_global_indices`
- `BatchPlanItem`
- `plan_batches`

- existing_completed_indices
- assert_sobol_config_compatible
- make_generation_summary
- print_generation_summary
- write_config_files
- run_preflight
- run_generation
- generate_batch
- write_generation_log
- write_sobol_index
- audit_dataset_batches
- write_validity_summary
- estimate_runtime_from_preflight

General LEBT dataset generation

- ParameterRange
- TwissRangeConfig
- EmittanceRangeConfig
- InitialOrbitRangeConfig
- BeamCurrentRangeConfig
- SteeringMagnetRangeConfig
- ElectrostaticQuadrupoleRangeConfig
- LEBTParameterRangeConfig
- default_lebt_parameter_ranges
- resolve_lebt_range_preset
- DatasetGenerationConfig
- LEBTDatasetConfig
- LEBTDryRunValidationResult
- validate_lebt_dry_run_config
- format_input_summary_table
- LEBTDatasetGenerationResult
- deterministic_seed
- generate_control_samples
- generate_input_samples
- generate_sobol_controls
- generate_lhs_controls
- generate_lebt_dataset
- beam_state_output_vector
- derive_twiss_from_state_moments
- validate_state_moment_labels

Input and target views

- InputViewSpec
- get_input_columns

- `select_input_view`
- `resolve_target_view_names`
- `load_target_view`

No-offset validation

- `ValidationIssue`
- `load_metadata`
- `load_json_file`
- `load_config_yaml`
- `discover_batch_files`
- `validate_required_files`
- `validate_config_values`
- `validate_sobol_index`
- `read_batch_global_indices`
- `check_fixed_zero_parameters`
- `check_output_dimensions`
- `check_diagnostic_rms_positive`
- `moments_to_twiss`
- `check_state_physics`
- `validate_batch_integrity`
- `validate_no_offset_sobol_dataset`
- `write_validation_reports`
- `format_validation_report_markdown`

Notebook Usage

`04_lebt_no_offset_sobol_dataset_production.ipynb` currently uses package code for:

- loading no-offset Sobol config
- displaying resolved config
- generation summary
- preflight execution
- optional preflight scaling runs
- production generation
- extension generation
- post-generation audit

Important safety behavior:

- full production is gated by `RUN_FULL_PRODUCTION = False`
- extension is gated by `RUN_EXTENSION = False`
- scale testing is gated by `RUN_PREFLIGHT_SCALE_TEST = False`

`05_lebt_no_offset_sobol_dataset_validation.ipynb` currently uses package code for:

- loading metadata/config/Sobol index

- validating required files
- validating batch integrity
- validating fixed-zero parameters
- validating diagnostics and beam-state physics
- writing validation reports
- optional plots

Preflight Path Inspection

The no-offset Sobol preflight path is available as:

`run_preflight(config)`

Current behavior:

- derives `preflight_dir = config.dataset_path / "preflight"`
- replaces the requested config with:
  - `target_total_samples = 240`
  - `mode = "preflight"`
  - `batch_size = 120`
  - `overwrite = True`
  - `resume = False`
- calls `run_generation(preflight, backend=backend)`

For this R0 report, the preflight path was inspected and config-loaded but not executed with TRACK. The loaded versioned configs reported:

```
production: dataset=lebt_production_no_offset_multitarget_sobol_v1, mode=production, samples
preflight:  dataset=lebt_production_no_offset_multitarget_sobol_v1, mode=preflight,  samples
dimensions: sampled=19, fixed=5, full=24, targets={diagnostics: 24, state_last_ws: 10, state
```

Duplicated Logic

The main duplication is between `dataset_generator.py` and `no_offset_sobol_production.py`:

- configuration validation
- deterministic seed/sample policy
- Sobol sampling
- parallel worker loops
- progress printing
- work directory cleanup
- HDF5 dataset writing
- metadata/config writing
- success/completed masks
- failure reason handling
- state output storage
- state moment/Twiss validation
- audit/report summaries

There is also some overlap between validation helpers and generation audit code:

- diagnostic finite checks
- RMS positivity checks
- state moment validity checks
- target dimension checks
- duplicate index checks

Current Test Coverage

Relevant tests exercised for this report:

```
python3 -m pytest tests/test_no_offset_sobol_production.py tests/test_no_offset_sobol_valida
```

Result:

```
76 passed
```

Full suite exercised:

```
python3 -m pytest -q
```

Result:

```
all tests passed, 3 skipped
```

Expected skips:

- CUDA test skipped because CUDA is not available.
- JuTrack integration skipped unless `SYNAPTICTRACK_RUN_JUTRACK=1`.
- Notebook execution skipped unless `SYNAPTICTRACK_RUN_NOTEBOOKS=1`.

Coverage strengths:

- Sobol determinism and continuation
- non-power-of-two Sobol range warning prevention
- duplicate global sample index detection
- batch resume and extension
- fixed-zero parameter placement
- dry-run 19D/24D/44D shapes
- target groups and failure handling
- validation report writing
- input view selection and ordering
- target view loading and combined target views
- state moment/Twiss physical checks on generated labels
- config load and invalid sample-count policy

Coverage gaps:

- production notebook execution is not run by default
- actual TRACK-backed preflight is not run in CI
- large production resume/extension is covered by small mock tests, not full-scale files

- performance/scaling behavior is notebook-driven and not unit-tested beyond config plumbing
- local machine override behavior is tested through temporary files, not the default ignored local path

Known Risky Areas

1. `no_offset_sobol_production.py` remains a large specialized orchestration module. Its responsibilities include sampling, config, execution, I/O, audit, resume, and reports.
2. `dataset_generator.py` and no-offset production generation still have overlapping execution machinery. This can cause future drift when adding MEBT or other accelerators.
3. HDF5 schema compatibility is critical. Existing notebooks/tests expect legacy and grouped names to keep working.
4. Global Sobol index semantics are high-risk. Production extension must never regenerate, duplicate, or reshuffle previous points.
5. Failed/lost-beam samples must remain explicit. Any refactor must preserve masks, `success`, `completed`, and `failure_reason`.
6. Beam-state moment physics checks are sensitive to numerical tolerances. Refactors should preserve `PHYS_TOL`-style near-zero handling.
7. Runtime paths and local config overrides are environment-sensitive. The production notebook must avoid hard-coded machine paths and must show the resolved config before generation.
8. The current no-offset Sobol workflow is specialized. Refactors should not make general accelerator abstractions depend on LEBT-specific names like `state_last_ws`.

Recommended Refactor Order

1. Preserve and expand schema centralization. Keep names, dimensions, masks, aliases, and target groups behind schema objects.
2. Extract config loading and validation policy. Keep production/preflight/test modes explicit and reject invalid production sample counts loudly.
3. Split no-offset Sobol production into focused modules:
    - sampling and Sobol index handling
    - batch planning and resume policy
    - batch HDF5 I/O
    - execution orchestration
    - audit/report generation

4. Extract shared generation runtime utilities from `dataset_generator.py` and `no_offset_sobol_production.py`:
    - worker loop
    - work directory policy
    - progress reporting
    - failure result normalization
5. Generalize accelerator/beamline schema support. Add future schemas without forcing MEBT or general accelerator workflows into LEBT-specific naming.
6. Move notebook logic steadily into package modules. Notebooks should load config, display config, call package functions, and visualize outputs.
7. Only after the above, consider broader API cleanup. Keep compatibility shims for existing notebooks/tests until replacement workflows are stable.

Acceptance Status

- Behavior changes: none introduced by this R0 report.
- Production notebook changes: none introduced by this R0 report.
- Production generation: not run.
- TRACK preflight generation: not run; path inspected and configs loaded.
- Relevant tests: passed.
- Full test suite: passed with expected skips.

Simulation Procedures and Production Dataset Workflow

Source Markdown: `docs/lebt_no_offset_sobol_workflow.md`

LEBT No-Offset Sobol Dataset Workflow

This is the maintained production workflow for the no-offset LEBT surrogate dataset:

```
lebt_production_no_offset_multitarget_sobol_v1
19D sampled input -> 44D target output
```

The notebook-facing API is `synapticTrack.simulation.no_offset_sobol_production`. Its public imports are maintained for production notebooks, while implementation details live in focused modules such as schema, sampling, batch planning, batch I/O, audit, and runtime helpers.

Beam current metadata uses canonical `uA` values and an explicit `track_current_per_synaptic_current` conversion factor. The default factor is `1.0` only for backward compatibility; TRACK current units remain unverified by local project documentation.

Fixed-Parameter Contract

The sampled input is 19D. A full 24D simulation input is accepted by the LEBT surrogate only when its fixed parameters match the dataset schema: `x0`, `xp0`, `y0`, and `yp0` are `0.0`, while `beam_current` is fixed at `0.05` in the current TRACK input convention. Full-input validation checks these expected values with a configurable tolerance; it does not treat every fixed parameter as zero.

Configure

Use the versioned config:

`configs/datasets/lebt_no_offset_sobol_production.yaml`

Put machine-specific paths in the local override file, which is git-ignored:

`configs/local/lebt_no_offset_sobol_local.yaml`

Typical local override fields:

```yaml
track_exe_path: TRACK/TRACKv39C.exe
source_dir: examples/lebt_orbit_corrections/orbit_corrections
calibration_dir: examples/lebt_orbit_corrections/orbit_corrections/calibrations
work_root: run.production/lebt_production_no_offset_multitarget_sobol_v1
```

Production Notebook

Run:

`notebooks/04_lebt_no_offset_sobol_dataset_production.ipynb`

Default behavior is safe:

```
RUN_PREFLIGHT = True
RUN_PREFLIGHT_SCALE_TEST = False
RUN_PRODUCTION_100K = False
RUN_EXTENSION_200K = False
RUN_EXTENSION_300K = False
RUN_EXTENSION_400K = False
RUN_EXTENSION_500K = False
RUN_EXTENSION_600K = False
RUN_EXTENSION_700K = False
RUN_EXTENSION_800K = False
RUN_EXTENSION_900K = False
RUN_EXTENSION_1000K = False
```

Only enable a production or extension stage after preflight passes. The parallel scale-test cell is optional because it launches multiple TRACK preflights and can select the fastest `n_workers` for staged production.

Sobol Extension

The supported production totals are:

```
100_000 -> 200_000 -> ... -> 900_000 -> 1_000_000
```

To extend, keep the same dataset name, Sobol dimension, scramble flag, and scramble seed. Enable one stage cell at a time:

```
RUN_PRODUCTION_100K = True
RUN_EXTENSION_200K = True
RUN_EXTENSION_300K = True
RUN_EXTENSION_400K = True
RUN_EXTENSION_500K = True
RUN_EXTENSION_600K = True
RUN_EXTENSION_700K = True
RUN_EXTENSION_800K = True
RUN_EXTENSION_900K = True
RUN_EXTENSION_1000K = True
OVERWRITE = False
RESUME = True
```

The generator plans missing global Sobol indices only. It must not reshuffle or regenerate completed samples.

Consolidated Stage Files

Generation writes resumable batch files under:

```
data/processed/lebt_production_no_offset_multitarget_sobol_v1/batches/
```

After a stage completes, you can optionally write one read-optimized HDF5 file for that stage without regenerating samples:

```
from synapticTrack.simulation.no_offset_sobol_production import consolidate_dataset_stage

for stage_total in range(100_000, 1_000_001, 100_000):
    consolidate_dataset_stage(config, target_total_samples=stage_total)
```

Default file names are:

```
data/processed/lebt_production_no_offset_multitarget_sobol_v1/lebt_production_no_offset_mult
...
data/processed/lebt_production_no_offset_multitarget_sobol_v1/lebt_production_no_offset_mult
```

The production notebook exposes `RUN_CONSOLIDATION` and `CONSOLIDATION_STAGE_TOTALS` for this step. Consolidation checks duplicate indices and requires a contiguous completed range from global Sobol index 0 to the requested stage total.

Validation Notebook

Run:

`notebooks/05_lebt_no_offset_sobol_dataset_validation.ipynb`

The notebook calls the pure audit API first:

```
from synapticTrack.data.production_audit import compute_validation_report
report = compute_validation_report(DATASET_DIR)
```

Reports are written only when requested:

```
WRITE_REPORTS = True
```

Outputs:

```
validation_report.json
validation_report.md
```

Training and Publication Metrics

The staged no-offset 44D surrogate workflow includes plain MLP and physics-regularized MLP training notebooks, model-comparison notebooks, and publication-metric table generation. Paper-oriented metrics should not use a single mixed-unit 10D state RMSE as the main physical metric. `compute_state_physics_metrics()` is the canonical report shared by plain and physics-regularized MLP evaluation. Use diagnostic RMSE in millimetres, component-wise state errors, normalized state errors, derived Courant-Snyder/emittance errors, and covariance-validity fractions. The mixed-unit value remains a marked legacy comparison metric only.

Current publication tables are generated under:

`models/lebt_no_offset_model_comparison/publication_metrics/`

The best diagnostics-only configuration from the available staged runs is the plain MLP at 1M samples. The physics-regularized 900k model is preferred when covariance-valid beam-state predictions are required.

API Status

Maintained production APIs:

```
from synapticTrack.simulation.no_offset_sobol_production import (
    load_no_offset_sobol_config,
    run_preflight,
    run_generation,
    make_generation_summary,
    plan_batches,
    audit_dataset_batches,
)
```

Maintained shared modules:

`synapticTrack.data.schema`
`synapticTrack.data.batch_io`
`synapticTrack.data.production_audit`
`synapticTrack.sampling.sobol`
`synapticTrack.sampling.latin_hypercube`
`synapticTrack.simulation.batch_planning`
`synapticTrack.simulation.generation_runtime`
`synapticTrack.utils.parallel`

Legacy/generic LEBT APIs in `synapticTrack.simulation.dataset_generator` are still supported for smoke datasets, dry runs, and development workflows. They should not be treated as the production no-offset Sobol pipeline.

CLI note

The current `synapticTrack` CLI `lebt-production-*` commands target the legacy/generic LEBT production-config path. They are not the maintained no-offset Sobol production workflow. Use the production notebook or package APIs above for `lebt_production_no_offset_multitarget_sobol_v1`.

Artifact And Dataset Compatibility

Each new training run writes `artifact_manifest.json` with format `training_run_artifacts_v2`. It records the stable paths for checkpoints, configuration, timing, plots, summaries, standard metrics, `state_physics_metrics_<split>.json`, compatibility `physics_metrics_<split>.json`, derived metrics, and covariance-validity metrics. Existing run directories without that file remain readable as an explicit legacy layout through `discover_run_artifacts()`.

No-offset loaders detect the HDF5 format before reading layout-specific paths. Current no-offset batch and consolidated formats are normalized through `read_hdf5_dataset_compat()`. See `docs/hdf5_dataset_compatibility.md` for supported legacy LEBT and MEBT formats and the future opt-in migration path.

Source Markdown: `docs/lebt_full_production_simulation_and_analysis.md`

LEBT Full Production Simulation and Analysis Runner

This document describes the one-file shell runner:

`scripts/run_lebt_full_production_and_analysis.sh`

The script runs the maintained no-offset Sobol LEBT production workflow and then runs dataset analysis/validation. It is intended for long full-production runs on a new machine, local workstation, Codex environment, or Antigravity prompt where screen output should be controlled.

What the Script Runs

The script is organized as a sequence of Python-backed execution stages. Each stage uses package APIs from `synapticTrack`; production logic is not duplicated inside notebooks or ad hoc scripts.

Stage	Shell label	Notebook section	Purpose
0	`00_manifest`	`00 Manifest`	Record run parameters, folders, Python/platform metadata, and local Git metadata.
1	`01_preflight`	`01 Preflight`	Run the 240-sample preflight using the selected scan and runtime options.
2	`02_production_stages`	`02 Production Stages`	Run cumulative Sobol production stages, normally `100000,200000,300000,400000,500000`.
3	`03_consolidation`	`03 Consolidation`	Write read-optimized consolidated HDF5 files for requested stages.
4	`04_analysis_validation`	`04 Analysis and Validation`	Compute final validation reports and write JSON/Markdown summaries.

Default Output Layout

Every execution uses a run-specific ID. If `--run-id` is not provided, the script uses a timestamp.

```
data/processed/full_production_runs/<run-id>/dataset/
run.production/full_production_runs/<run-id>/work/
run.production/full_production_runs/<run-id>/logs/
```

This keeps simulation outputs, temporary TRACK work files, and logs in separate folders. Override these with `--dataset-dir`, `--work-root`, and `--log-dir`.

Parallel Execution

By default, the script computes the number of online CPU cores with:

```
getconf _NPROCESSORS_ONLN
```

It then uses 90 percent of those cores for production workers. Override this with either:

```
--n-workers 32
--cpu-percent 75
```

`--n-workers` takes precedence over `--cpu-percent`.

Verbose and Quiet Modes

Quiet mode is the default. It disables sample progress and TRACK verbosity on screen and redirects each Python stage into a run-specific log file. This is the recommended mode for Codex or Antigravity prompt execution because it avoids large token consumption during full simulation.

```
scripts/run_lebt_full_production_and_analysis.sh --quiet
```

Verbose mode prints progress on screen and uses the requested TRACK verbosity:

```
scripts/run_lebt_full_production_and_analysis.sh --verbose --track-verbose 1
```

Full Production Example

```
scripts/run_lebt_full_production_and_analysis.sh \
  --run-id lebt_no_offset_500k_v1 \
  --stages 100000,200000,300000,400000,500000 \
  --quiet
```

The script applies `configs/local/lebt_no_offset_sobol_local.yaml` by default if that file exists. Use it for machine-specific TRACK paths and source directories. To ignore local overrides:

```
scripts/run_lebt_full_production_and_analysis.sh --no-local-override
```

Test-Sized Pipeline Check

Use `--dry-run` and `--allow-nonstandard` for a small local check that does not call TRACK:

```
scripts/run_lebt_full_production_and_analysis.sh \
  --run-id dry_run_240 \
  --stages 240 \
  --batch-size 120 \
  --n-workers 2 \
  --dry-run \
  --allow-nonstandard \
```

```
  --skip-preflight \
  --quiet
```

Analysis Outputs

The analysis stage writes the maintained validation reports into the dataset directory:

```
validation_report.json
validation_report.md
final_analysis_summary.json
```

Generation also writes production metadata and audit files, including:

```
run_manifest.json
generation_log.csv
sobol_index.json
audit_report.json
validity_summary.json
```

Matching Notebook

The matching notebook is:

`notebooks/20_lebt_full_production_simulation_and_analysis.ipynb`

It uses the same parameter names, output folder layout, stage order, and package APIs as the shell runner. Use the shell script for unattended full production and the notebook for step-by-step inspection, review, and manuscript-supporting analysis notes.

Repository Operation Reminder

Do not run `git push` unless the repository owner explicitly requests it. Remote GitHub Actions should be checked only when explicitly requested for a CI/debugging task. Local GitHub Actions workflow files may be edited in the local repository when needed.

Analysis Methods and Analysis Procedure

Source Markdown: `docs/scanner_measurement_analysis.md`

Scanner Measurement Analysis

This note documents how scanner measurement data is manipulated by the current analysis path in `synapticTrack`.

Relevant files:

- `src/synapticTrack/io/scanner_io.py`

- src/synapticTrack/beam/beam_scanner.py
- src/synapticTrack/analysis/scanner_analysis.py
- src/synapticTrack/utils/stats.py
- src/synapticTrack/utils/math_functions.py

File Loading

Scanner files are read with whitespace-separated columns and one skipped header row.

Wire scanner files are read as:

```
x_pos x_current y_pos y_current d_pos d_current
```

The required analysis columns are:

```
x_pos, x_current, y_pos, y_current
```

Allison scanner files are read as:

```
x xp x_current hv y_current
```

The required analysis columns are:

```
x, xp, x_current
```

The file stem is stored as `scan_id`.

Wire Scanner Manipulation

Wire scanner analysis is implemented by `analyze_wire_scanner`.

Each plane is processed independently:

```
x_pos, x_current
y_pos, y_current
```

The preprocessing function is `_preprocess_profile(position, current, shift, cutoff, cutoff_frac)`.

Processing steps:

1. Convert `position` and `current` to floating-point NumPy arrays.
2. Require matching shapes.
3. Remove all samples where either position or current is NaN or Inf.
4. If `shift=True`, subtract the minimum current from the current profile:

   ```
   current <- current - min(current)
   ```

   This is a baseline shift. It makes the smallest remaining current value zero.
5. If `cutoff=True`, remove low-current samples after the optional shift:

```
threshold = cutoff_frac * max(current)
keep current >= threshold
```

6. Require at least 4 points after filtering.
7. Require positive total current weight:

   ```
   sum(current) > 0
   ```

Default behavior:

```
shift=False
cutoff=False
cutoff_frac=0.05
```

So by default the analysis removes non-finite rows only. Baseline subtraction and cutoff filtering are opt-in.

Wire Scanner Derived Quantities

After preprocessing, weighted centroid and RMS size are computed with current as the weight.

For one plane:

```
center = sum(current_i * position_i) / sum(current_i)
rms    = sqrt(sum(current_i * (position_i - center)^2) / sum(current_i))
```

The returned weighted quantities are:

```
x_center
y_center
sigma_x
sigma_y
```

A separate Gaussian fit is also performed for each plane:

```
current(position) = a * exp(-(position - x0)^2 / (2 * sigma^2)) + offset
```

Initial fit guesses are:

```
a      = max(current)
x0     = weighted center
sigma  = weighted rms
offset = 0
```

The returned fit quantities are:

```
gaussian_fit_x_center
gaussian_fit_y_center
gaussian_fit_sigma_x
gaussian_fit_sigma_y
```

The function also returns metadata about preprocessing:

```
shift_applied
cutoff_applied
cutoff_fraction
x_points_used
y_points_used
```

If plotting is enabled, the plot uses the processed profiles, not the original raw profiles.

Allison Scanner Manipulation

Allison scanner analysis is implemented by `analyze_allison_scanner_2d`.

Current behavior uses the loaded arrays directly:

```
x
xp
x_current
```

Unlike wire scanner analysis, there is currently no explicit finite filtering, baseline shifting, or cutoff filtering in `analyze_allison_scanner_2d`.

Weighted centroids and RMS values are computed using `x_current` as the weight:

```
x_center  = weighted mean of x
xp_center = weighted mean of xp
sigma_x   = weighted RMS of x
sigma_xp  = weighted RMS of xp
```

The weighted covariance is:

```
covariance_x_xp =
    sum(x_current_i * (x_i - x_center) * (xp_i - xp_center))
    / sum(x_current_i)
```

The RMS emittance is:

```
emittance_rms = sqrt(sigma_x^2 * sigma_xp^2 - covariance_x_xp^2)
```

The current geometric-emittance estimate is:

```
emittance_geometric = sigma_x * sigma_xp
```

Separate 1D Gaussian fits are performed for `x` and `xp` projections using the same current weights as measured values:

```
x_current(x)  = a * exp(-(x  - x0)^2  / (2 * sigma_x^2))  + offset
x_current(xp) = a * exp(-(xp - xp0)^2 / (2 * sigma_xp^2)) + offset
```

Initial fit guesses are:

```
a      = max(x_current)
center = weighted center
```

```
sigma  = weighted RMS
offset = 0
```

Returned Allison quantities:

```
x_center
xp_center
sigma_x
sigma_xp
gaussian_fit_x_center
gaussian_fit_xp_center
gaussian_fit_sigma_x
gaussian_fit_sigma_xp
covariance_x_xp
emittance_rms
emittance_geometric
```

If plotting is enabled, the plot uses the original loaded Allison arrays.

Important Assumptions And Caveats

- Wire scanner preprocessing is optional except for NaN/Inf removal.
- Allison scanner analysis currently does not remove NaN/Inf values before computing moments or fitting.
- Current values are used directly as weights. Negative current values are not clipped unless `shift=True` is used for wire scanner profiles.
- Wire scanner Gaussian-fit sigmas are returned directly from `curve_fit`; the code does not force them positive with `abs`.
- The RMS moment calculation and Gaussian fit are separate estimates. Downstream code should be explicit about which one it uses.
- Plot files reflect processed wire-scanner data when preprocessing is enabled.

Source Markdown: `docs/notebook_hygiene.md`

Notebook Hygiene

Committed notebooks should contain source cells only:

- no execution counts;
- no embedded outputs;
- no absolute machine-specific paths;
- no local TRACK paths in source cells.

Use package code from `src/synapticTrack`; do not redefine production functions inside notebooks.

Covered Notebook Roots

Notebook hygiene tests cover notebooks in `examples/`, `notebooks/`, and `RAON/`. This includes mirrored RAON LEBT notebooks, extended LEBT training notebooks, and MEBT TRACK notebooks. Generated run folders and scratch/output files should be ignored locally rather than committed.

Clear Outputs

Recommended workflow before committing notebooks:

```
python3 scripts/strip_notebook_outputs.py
python3 scripts/strip_notebook_outputs.py --check
```

By default, the script processes `examples/`, `notebooks/`, and `RAON/`, matching the notebook roots covered by the tests, and skips `.ipynb_checkpoints`. Pass explicit files or directories to limit the scope:

```
python3 scripts/strip_notebook_outputs.py RAON/MEBT_notebooks
```

`nbstripout` is also listed in `pyproject.toml` and can be installed as a git filter, but the project script is dependency-free and covers all notebook roots checked by the tests.

Local Paths

Use config files and environment variables for machine-specific paths. For example:

```
TRACK_EXE_PATH = Path(os.environ.get('TRACK_EXE_PATH', '../TRACK/TRACKv39C.exe')).expandusei
```

For production Sobol generation, prefer `configs/local/lebt_no_offset_sobol_local.yaml`; files under `configs/local/` are git-ignored except examples.

Checks

The test suite includes notebook hygiene checks for project notebooks:

```
python3 -m pytest tests/test_notebooks.py -q
```

New User Installation and Local Operation

Source Markdown: `docs/INSTALL.md`

Installation Guide

This guide is for a new user setting up `synapticTrack` on a new machine.

Prerequisites

- Git
- Python 3.8 or newer
- A working C/C++ build toolchain if your platform needs to compile Python wheels
- Network access to the Python package index used by `pip`

`synapticTrack` is a Python package. Install it from the repository root with `pip install -e .`; the project dependencies are declared in `pyproject.toml`.

Quick Install

Replace `<repo-url>` with the actual Git URL for this repository:

```
git clone <repo-url> synapticTrack
cd synapticTrack

python3 -m venv .venv
source .venv/bin/activate

python -m pip install --upgrade pip setuptools wheel
python -m pip install -e .
python -m pytest
```

If your system uses `python` instead of `python3`, substitute that command in the `venv` creation step.

Scripted Install

The repository includes a bootstrap script that performs the same steps:

```
bash scripts/bootstrap_install.sh <repo-url>
```

By default, the script clones into `synapticTrack`. To choose a different target directory:

```
bash scripts/bootstrap_install.sh <repo-url> /path/to/synapticTrack
```

The script:

1. checks that Git and Python are available;
2. clones the repository if the target directory does not already exist;
3. creates a virtual environment at `.venv`;
4. installs the package in editable mode;
5. runs the full pytest suite.

Reusing an Existing Clone

If the target directory already exists, the script does not reclone it. It reuses the existing checkout and continues with virtual environment creation, package

installation, and tests:

```
bash scripts/bootstrap_install.sh <repo-url> /existing/path/to/synapticTrack
```

Dependency Metadata

Continuous integration installs the package with:

```
python -m pip install -e .[dev]
```

The install metadata in `pyproject.toml` and `setup.py` is therefore the source of truth for CI dependencies. `PyYAML` is declared there because the maintained configuration loaders import `yaml` for local and production dataset configuration files.

Optional External TRACK Setup

Some workflows require the external TRACK executable, which is not committed to this repository. See `docs/TRACK_SETUP.md` after the Python package is installed.

Verify Console Commands

After installation, these package commands should be available inside the activated virtual environment:

```
synapticTrack --help
syntrack-track-check --help
syntrack-lebt-preflight --help
syntrack-lebt-validate --help
```

Development and Agent Operating Instructions

Source Markdown: `CODEX.md`

synapticTrack

Overview

synapticTrack is a Python framework for accelerator modeling, beam dynamics analysis, machine learning, optimization, and accelerator control applications.

The framework is intended to support both simulation and future online accelerator operation.

Primary application areas include:

- Beam dynamics simulations
- Accelerator lattice modeling
- Beam diagnostics analysis

- Scanner data processing
- Machine learning surrogate modeling
- Reinforcement learning based control
- Multi-objective optimization
- Bayesian optimization
- Digital twin development
- EPICS/TANGO integration
- Accelerator AI applications

Current accelerator applications include:

- RAON LEBT orbit correction
- RAON MEBT studies
- Electron injector optimization
- Space-charge studies
- Beam phase-space reconstruction
- Accelerator tuning and control

Software Architecture

Core Modules

beam Beam representations:

- Beam
- BeamWS
- BeamAS
- Twiss

Responsibilities:

- Particle coordinates
- Beam statistics
- Centroids
- RMS sizes
- Emittances
- Twiss parameters

---

io Input/output interfaces:

- TRACK
- JuTrack
- OPAL
- FLAME
- User-defined formats

Responsibilities:

- Read beam files
- Write beam files
- Conversion between simulation codes

---

lattice Accelerator lattice definitions.

Responsibilities:

- Element models
- TRACK lattice generation
- Lattice manipulation
- Future MAD-X support

---

analysis Beam diagnostics and data analysis.

Responsibilities:

- Wire scanner analysis
- Allison scanner analysis
- Centroid extraction
- RMS calculations
- Emittance calculations
- Beam quality metrics

---

visualization Plotting and diagnostics.

Responsibilities:

- Phase-space plots
- Beam distributions
- Orbit plots
- Scanner plots
- Optimization histories

---

optimization Optimization algorithms.

Current and planned methods:

- Differential Evolution
- Genetic Algorithms
- Bayesian Optimization

- Multi-objective Optimization
- Reinforcement Learning

---

machine_learning Data-driven accelerator modeling.

Current methods:

- Fully Connected Neural Networks
- Surrogate Models

Planned methods:

- Physics-Informed Neural Networks
- Graph Neural Networks
- Transformer Models
- Differentiable Beam Dynamics

---

reinforcement_learning Closed-loop accelerator control.

Current methods:

- PPO

Planned methods:

- SAC
- TD3
- DDPG
- Model-based RL

Applications:

- Orbit correction
- Beam tuning
- Autonomous operation

Development Principles

Avoid Circular Imports Do not import high-level modules inside low-level modules.

Preferred dependency direction:

beam → io → analysis → visualization

optimization → machine_learning → reinforcement_learning

Avoid:

analysis → reinforcement_learning

reinforcement_learning → analysis

Use local imports when necessary.

Public APIs Do not break existing public APIs unless explicitly requested.

Maintain backward compatibility whenever possible.

Testing Run before every commit:

python -m pytest tests -v

For import-related changes:

python -c “import synapticTrack”

python -c “import synapticTrack.utils.sm_utils”

python -c “import synapticTrack.utils.rl_utils”

Machine Learning Conventions RAON LEBT orbit correction:

Inputs:

[FH1,FH2,FH3,FH4,FH5, FV1,FV2,FV3,FV4,FV5, alpha1,alpha2,alpha3]

Outputs:

[WS1_x, AS_x, WS2_x, WS3_x, WS4_x, END_x, WS1_y, AS_y, WS2_y, WS3_y, WS4_y, END_y]

Steering magnet units:

mrad

Alpha parameters:

dimensionless scaling factors

Code Quality Prefer:

- Small functions
- Unit tests
- Explicit typing
- Clear documentation

Avoid:

- Hidden global state
- Wildcard imports
- Circular dependencies
- Hard-coded paths

Source Markdown: `AGENTS.md`

AGENTS.md — synapticTrack AI/ML Development

Scope

Implement the staged 2026 program for LEBT surrogate modeling, Gaussian beam-state inference, MEBT surrogate modeling, RL orbit correction, and later advanced reconstruction.

Required principles

- Use PyTorch for ML and differentiable inference.
- The core initial transverse distribution is an uncoupled 4D Gaussian.
- Do not add a separate shape latent vector to the core model.
- LEBT inputs: Twiss parameters, emittances, offsets, angles, current, steering strengths, and electrostatic-quadrupole strengths.
- LEBT outputs: centroids, rms sizes, and beam states at the last WS and LEBT exit.
- Build the MEBT surrogate before RL orbit correction.
- Include injected beam state in the MEBT surrogate inputs.
- Put reusable logic in Python modules; notebooks import package code.

Scientific rules

- State units explicitly and reuse existing synapticTrack conventions.
- Validate positive beta, positive emittance, and non-negative current.
- Preserve covariance positive definiteness.
- Store failed/lost-beam cases with validity masks and reasons.
- Use deterministic seeds and record code/configuration metadata.

Testing

Every milestone adds tests for relevant physics, tensor shapes, dtype/device behavior, determinism, serialization, bounds, reward behavior, and termination conditions. Run the relevant existing tests before completing a task.

Codex behavior

- Inspect existing code before adding modules.
- Reuse existing beam and backend abstractions.

- Implement only the requested milestone.
- Avoid unrelated refactors.
- Report assumptions, changed files, commands, test outcomes, and blockers.

Repository operations

- Do not run `git push`.
- Do not check GitHub Actions or remote CI status.
- Local GitHub Actions workflow files may be modified when requested or needed, but only in the local repository.

Reporting format

Every completed task summary must explicitly include:

- assumptions;
- changed files;
- commands run;
- test or validation outcomes;
- blockers or limitations;
- confirmation that no `git push` was run and remote GitHub Actions were not checked.

Project attribution

- Author: Chong Shik Park.
- Affiliation: Department of Accelerator Science and Center for Accelerator Research, Korea University, Sejong, 30019 Republic of Korea.

Unclassified Markdown Sources

Source Markdown: `docs/beam_state_physics_api.md`

Beam-State Physics API

`synapticTrack.beam.state_physics` is the canonical implementation for the LEBT/MEBT 10D beam state:

`x_c, xp_c, y_c, yp_c, <x^2>, <xx'>, <x'^2>, <y^2>, <yy'>, <y'^2>`

The module centralizes component units and determinant tolerances. Use `state10_physics(state, mode="differentiable")` in PyTorch losses: it applies one finite determinant floor to preserve gradients. Use `state10_physics(state, mode="strict")` for validation and publication metrics: invalid covariance-derived quantities remain `NaN` and `validity` reports finite-input, determinant, emittance, beta, and overall validity flags.

`state10_from_cholesky_parameters` provides the explicit full-state positive-definite reconstruction mode; `covariance_from_cholesky_params` and related functions remain available for individual planes. New code should import this module from `synapticTrack.beam`. `synapticTrack.ml.beam_physics`, `ml.metrics.moments_to_twiss`, `data.production_audit.moments_to_twiss`, and `simulation.dataset_generator.derive_twiss_from_state_moments` remain compatibility wrappers and should not be used in new code.

Publication and audit reports must not clamp invalid determinants. They retain invalid values as `NaN`, count them explicitly, and omit them from derived error aggregates while recording `n_invalid`.

Metric Reports

`compute_state_physics_metrics()` in `synapticTrack.ml.metrics` is the canonical strict reporting API for `state_last_ws` and `state_lebt_exit`. Its per-group output includes component and normalized errors, physical-class metrics, derived Courant-Snyder/emittance errors, covariance-validity fractions, warnings, sample counts, and the recorded tolerance configuration.

Physics-regularized training writes a compatibility `physics_metrics_<split>.json` report in addition to canonical metrics. It preserves historical flat fields, but new consumers should read the canonical `groups` report or `state_physics_metrics_<split>.json`. Mixed-unit state RMSE is secondary and must not be interpreted as a physical acceptance criterion.

Source Markdown: `docs/codex_refactor_tasks/01_fixed_parameter_inference_contract.md`

Codex Prompt: Fixed-Parameter Inference Contract

Read `AGENTS.md`, the current LEBT no-offset schema, surrogate API, and tests.

Task

Refactor full 24D LEBT surrogate input validation so it checks each fixed parameter against its schema-defined fixed value rather than assuming every fixed parameter is zero.

`beam_current` is fixed at `0.05` in the current no-offset production schema; the injection coordinates remain zero.

Requirements

1. Replace the public `fixed_zero_tol`/`allow_nonzero_fixed` terminology with names that reflect validation against fixed expected values. Preserve a backward-compatible API where practical.
2. Keep accepting the sampled 19D input directly.
3. For full 24D inputs, compare `x0`, `xp0`, `y0`, `yp0`, and `beam_current` against `LEBT_FIXED_PARAMETER_VALUES` using a configurable tolerance.

4. Reject an input whose fixed current differs from `0.05`, while accepting a valid full row with that current by default.
5. Ensure errors identify the parameter, expected value, observed maximum error, and tolerance.
6. Reuse the same fixed-value helper in production audit code where possible; do not maintain two independent fixed-value rules.
7. Update affected notebooks and docs that still say "fixed zero" when they mean fixed injection/current.

Tests

Add or update tests for valid full 24D input with current `0.05`, rejection of incorrect current, rejection of nonzero orbit offsets, direct 19D input, and backward-compatible public arguments.

Do not modify generated datasets or retrain models.

Source Markdown: `docs/codex_refactor_tasks/02_beam_current_unit_contract.md`

Codex Prompt: Beam-Current Unit Contract

Read `AGENTS.md`, TRACK input handling, Gaussian beam-state code, LEBT/MEBT production configs, and relevant documentation.

Task

Create one explicit, tested beam-current unit contract between synapticTrack and TRACK.

The code currently labels beam current as `uA`, while TRACK templates contain `current = 0.05`, and injection writes values directly to `track.dat`.

Requirements

1. Verify TRACK's `current` unit from project documentation or the TRACK reference available locally. If it cannot be verified locally, do not guess: define a clearly marked configuration boundary and document the unresolved external assumption.
2. Define canonical synapticTrack and TRACK units in one reusable module.
3. Add explicit conversion functions with finite/non-negative validation.
4. Make `update_track_dat_twiss()` use the conversion rather than copying a raw value.
5. Ensure `GaussianBeamState`, `Beam`, `DEFAULT_LEBT_UNITS`, `TRACK_BEAM_METADATA`, LEBT production, and MEBT production agree on the contract.
6. Store both numeric value and unit in generated metadata. Preserve existing files and provide compatibility handling for metadata without units.
7. Update documentation and paper-facing text only after the unit is verified.

Tests

Test conversion round trips, TRACK-file replacement, non-negative rejection, LEBT fixed current, MEBT fixed current, and output-beam metadata propagation.

Do not silently reinterpret existing production data.

Source Markdown: `docs/codex_refactor_tasks/03_mebt_surrogate_input_schema.md`

Codex Prompt: MEBT Surrogate Input Schema

Read `AGENTS.md`, MEBT Sobol production code, LEBT schema abstractions, and the MEBT notebook workflow.

Task

Define reusable MEBT dataset and ML schemas that include injected beam state in the surrogate input contract.

The current MEBT Sobol workflow samples 19 lattice controls and starts from a fixed `scratch.#07` distribution. That is useful for a fixed-distribution diagnostic study, but not for the stated MEBT surrogate requirement.

Requirements

1. Keep the current fixed-distribution 19D dataset format valid and explicitly identify it as such.
2. Add a versioned MEBT schema for a general surrogate with:
   - lattice controls;
   - injected transverse beam state, using the established 10D state-moment convention or a documented equivalent physical parameterization;
   - fixed-current value and unit metadata;
   - validity masks and failure reasons.
3. Reuse `DatasetSchema`/ML schema patterns rather than defining a notebook-only format.
4. Define target groups and units, including diagnostics and any requested MEBT beam states.
5. Separate sampling bounds for controls and injected state, validating positive beta/emittance and positive-definite covariance.
6. Document how `scratch.#07` becomes a nominal reference distribution versus a per-sample injected state.
7. Do not change the 44D LEBT output format or run a production campaign.

Tests

Test schema dimensions/names, deterministic sample expansion, valid state acceptance, invalid covariance rejection, metadata serialization, and compatibility

loading of the current fixed-distribution MEBT batches.

Source Markdown: `docs/codex_refactor_tasks/04_physics_training_output_scaling.md`

Codex Prompt: Physics-Regularized Output Scaling

Read `AGENTS.md`, Task 4/Task 5 training modules, normalizers, loss functions, and the physics-regularized training configuration.

Task

Make output scaling a real, reproducible part of physics-regularized training.

The current physics model trains against physical-space targets directly, even though output normalizers are fitted and `output_normalization` is configurable. This leaves mixed physical scales to dominate data and derived losses.

Requirements

1. State clearly which quantities are predicted in normalized coordinates and which physics losses must be evaluated after conversion to physical units.
2. Introduce a reusable differentiable output transform based on train-split statistics. It must work on CPU/GPU and preserve dtype/device.
3. Apply scaling consistently to diagnostics, state components, Courant-Snyder loss, covariance penalties, and evaluation output.
4. Make `output_normalization` either operational or remove it with a documented migration path; it must not remain a no-op configuration field.
5. Preserve checkpoint loading and the public `predict_physical()` API.
6. Record output scaling values, source split, and transform version in model artifacts.
7. Avoid changing model architecture or retraining existing production models.

Tests

Test normalizer round trips, differentiable transforms, physical-space metric consistency, component balancing for disparate scales, checkpoint reload, and deterministic small training runs.

Source Markdown: `docs/codex_refactor_tasks/05_unify_beam_state_physics_api.md`

Codex Prompt: Unify Beam-State Physics API

Read `AGENTS.md` and the current beam-physics, metrics, production-audit, physics-training, inference, and beam modules.

Task

Consolidate the duplicated 10D state-moment, Courant-Snyder, emittance, and covariance-validity calculations into one documented API supporting NumPy

and PyTorch.

Requirements

1. Establish one canonical state order: `x_c, xp_c, y_c, yp_c, <x^2>, <xx'>, <x'^2>, <y^2>, <yy'>, <y'^2>`.
2. Define explicit modes for:
   - differentiable training calculations, with safe finite gradients;
   - strict validation/reporting, with invalid values represented as `NaN` and associated validity flags;
   - positive-definite Cholesky reconstruction.
3. Centralize determinant thresholds and unit definitions. Do not let each caller choose unrelated defaults.
4. Migrate `ml.metrics`, `data.production_audit`, `ml.physics_training`, and surrogate inference to the shared implementation.
5. Preserve existing metric JSON keys where needed, but deprecate duplicate helpers and document replacements.
6. Ensure derived metrics count non-finite values explicitly and never produce silently misleading aggregate errors.

Tests

Add cross-module consistency tests for valid/invalid covariances, NumPy/Torch agreement, gradients, all 10D components, both LEBT state groups, and JSON-safe reporting.

Do not change the physical definitions or silently clamp invalid values in publication metrics.

Source Markdown: `docs/codex_refactor_tasks/06_legacy_physics_cleanup.md`

Codex Prompt: Legacy Physics and Runtime Cleanup

Read `AGENTS.md`, legacy beam utilities, TRACK runtime helpers, and their tests.

Task

Repair known legacy correctness issues and remove brittle assumptions while keeping public behavior compatible.

Requirements

1. Fix `beam.twiss.Twiss.compute_twiss()` so its self-consistency warning does not reference undefined `label`.
2. Route legacy validity summaries through the canonical state-physics API from the beam-state refactor. Non-finite beta/emittance values must be counted as invalid, not omitted by comparisons with `NaN`.

3. Make MEBT lattice segment discovery order numeric suffixes, so `elements10.json` follows `elements9.json`.
4. Replace LEBT-specific TRACK runtime defaults such as `scratch.#02` with a small runtime specification object or validated arguments. Keep existing defaults for callers.
5. Replace print-and-continue output-file handling with structured warnings or results where failures affect downstream correctness.
6. Add focused regression tests for each repaired path.

Do not remove legacy public functions abruptly. Keep compatibility shims and mark them deprecated where appropriate.

Source Markdown: `docs/codex_refactor_tasks/07_track_execution_orchestration.md`

Codex Prompt: Unify TRACK Execution Orchestration

Read `AGENTS.md`, `utils/track_runtime.py`, `track/run_track.py`, `simulation/lebt_track_adapter.py`, `simulation/mebt_sobol_production.py`, and their tests.

Task

Create one reusable, path-safe TRACK segmented-execution service for LEBT and MEBT. Remove global-current-working-directory dependence from legacy execution while preserving existing public functions as compatibility wrappers.

Motivation

`run_calibrated_track()` changes the process-wide working directory, while the LEBT adapter and MEBT Sobol producer independently prepare runtimes, write segments, hand off scratch distributions, collect outputs, and clean work directories. This makes concurrent execution and failure handling brittle.

Requirements

1. Add a typed execution request/result API in a reusable runtime module. It must include source/work directories, `TrackRuntimeSpec`, lattice segments, output directories, TRACK executable, verbosity, and the initial scratch index.
2. Execute every path explicitly. Do not call `os.chdir()` in library code.
3. Keep the existing scratch handoff contract: a segment loading `scratch.#NN` must make `scratch.#(NN+1)` available for the next segment.
4. Centralize preparation, segmented execution, final-output collection, and cleanup policy. MEBT must retain `scratch.#07` as its configured initial distribution; LEBT retains its default `scratch.#02`.
5. Make `run_calibrated_track()` and `run_track_steps()` compatibility wrappers around the new path-safe API where practical. Do not break notebook callers.

6. Have the LEBT adapter and MEBT producer use the shared service rather than maintaining independent orchestration sequences.
7. Return structured execution data including segment outputs, final directory, scratch handoff records, and cleanup outcome. Preserve existing numerical diagnostic returns through wrappers.

Tests

Add tests for:

1. current working directory is unchanged on success and exceptions;
2. LEBT default initial scratch and MEBT configured initial scratch;
3. multi-segment scratch handoff in numeric order;
4. absolute and relative output paths;
5. cleanup policies for successful and failed runs;
6. compatibility of existing `run_calibrated_track()` and adapter APIs.

Do not invoke the real TRACK executable in tests. Do not retrain models or change dataset formats.

Source Markdown: `docs/codex_refactor_tasks/08_track_diagnostic_output_contract.md`

Codex Prompt: Enforce TRACK Diagnostic Output Contracts

Read `AGENTS.md`, `utils/track_outputs.py`, TRACK output collection code, LEBT/MEBT production schemas, and their tests.

Task

Define one explicit, validated contract for segmented TRACK diagnostic outputs and use it for LEBT and MEBT collection.

Motivation

Current helpers use `zip()` and positional indexing when copying coordinate files and combining `z_scanner.out`, centroids, and RMS values. A missing or extra path can silently truncate collection or produce a misaligned diagnostic vector.

Requirements

1. Add typed diagnostic-location and output-spec objects containing the diagnostic name, segment output path, final coordinate filename, and unit.
2. Validate exact cardinality, unique final filenames, readable coordinate files, and exact z-position alignment before creating output vectors.
3. Replace positional `zip()` truncation in output-copy helpers with explicit length checks and informative errors.

4. Validate output arrays are finite and all RMS quantities are positive for samples marked valid. Preserve lost/failed samples with masks and reasons.
5. Return a structured diagnostic collection result with source paths, z positions, centroids, RMS values, units, and validation findings.
6. Retain `get_beam_data()`, `diagnostics_to_output_vector()`, and existing copy helpers as documented compatibility wrappers.
7. Reuse schema names and units instead of duplicating ordered strings in LEBT and MEBT producers.

Tests

Add tests for:

1. valid LEBT and MEBT diagnostic layouts;
2. unequal segment/output/final-file lengths;
3. duplicate final filenames;
4. missing coordinate output and missing/misaligned z-scanner entries;
5. nonfinite centroid and nonpositive RMS handling;
6. legacy helper output equivalence for valid inputs.

Do not run real TRACK or modify generated datasets.

Source Markdown: `docs/codex_refactor_tasks/09_production_batch_atomicity_and_failure_taxonomy`

Codex Prompt: Atomic Production Batches and Failure Taxonomy

Read `AGENTS.md`, LEBT/MEBT Sobol production modules, batch planning, HDF5 I/O, generation runtime helpers, and tests.

Task

Make staged LEBT and MEBT sample production resumable and auditable under worker, TRACK, and filesystem failures.

Motivation

MEBT workers currently serialize broad exceptions as unstructured strings and write batch HDF5 files directly. Interrupted writes, malformed partial batches, and different failure representations across LEBT/MEBT can corrupt resume and audit decisions.

Requirements

1. Define a reusable typed sample outcome with global index, validity mask, failure category, human-readable reason, optional exception class, runtime, and work-directory disposition.

2. Establish a small stable failure taxonomy, including input validation, runtime preparation, TRACK process, required output, parsing, and internal errors. Preserve original messages as context.
3. Write batches to a same-filesystem temporary path, flush/close successfully, then atomically replace the final HDF5 path. Set `batch_completed` only after durable completion.
4. Make resume/audit reject partial, unreadable, schema-incomplete, or non-contiguous batches with actionable findings; never treat them as complete.
5. Keep valid/lost/failed rows and reasons in the dataset rather than silently dropping them. Maintain deterministic Sobol index ordering in serial and multiprocess paths.
6. Preserve existing HDF5 fields and add new metadata/fields compatibly with explicit schema-version handling.
7. Share the outcome and batch-commit logic between LEBT and MEBT where their contracts overlap.

Tests

Add tests for:

1. atomic success commit and absence of temporary files afterward;
2. simulated write failure leaves no completed final batch;
3. corrupt or incomplete batch rejection during resume;
4. every failure category and JSON/HDF5-safe reason storage;
5. serial/multiprocess ordering equivalence using a fake backend;
6. backward-compatible reading of existing completed batches.

Do not invoke TRACK, delete user production data, or retrain models.

Source Markdown: `docs/codex_refactor_tasks/10_versioned_artifact_and_config_loading.md`

Codex Prompt: Versioned Artifact and Configuration Loading

Read `AGENTS.md`, `ml/io_utils.py`, production configuration discovery, checkpoint/normalizer artifact loading, and tests.

Task

Replace ambiguous silent artifact/config parsing with explicit loading results and numeric version-aware discovery, while preserving the current convenience APIs.

Motivation

Several readers return `None` or an empty list for both a missing file and a malformed file. Versioned configuration discovery also relies on lexical path

sorting, which misorders versions such as `v10` and `v2`. These behaviors can cause training or production to run with fallback defaults without a visible root cause.

Requirements

1. Add a reusable artifact-load result or typed exception contract that distinguishes missing, invalid syntax, invalid schema, unsupported version, and successful load.
2. Add strict reader variants for JSON, YAML, CSV, normalizer manifests, and production configs. Existing tolerant helpers must remain available and be documented as compatibility behavior.
3. Parse versioned filenames numerically and deterministically. Reject ambiguous names and duplicate logical versions with actionable errors.
4. Validate minimum required metadata before an artifact is consumed: schema version, dimensions where applicable, units/current-contract metadata, and source/config identity.
5. Record the resolved artifact paths and validation result in training and production manifests, without modifying historical artifact contents.
6. Remove broad exception swallowing where an invalid artifact would otherwise be mistaken for a missing optional artifact.

Tests

Add tests for:

1. missing versus malformed JSON/YAML/CSV;
2. numeric ordering of `v1`, `v2`, and `v10` configurations;
3. duplicate and malformed versioned filenames;
4. strict metadata rejection and tolerant compatibility behavior;
5. checkpoint/normalizer manifest path resolution;
6. JSON-safe load diagnostics.

Do not change trained model weights, regenerate datasets, or require remote resources.

Source Markdown: `docs/codex_refactor_tasks/11_mebt_batch_schema_and_fixed_current_contract.md`

Codex Prompt: MEBT Batch Schema and Fixed-Current Contract

Read `AGENTS.md`, the MEBT Sobol producer, `DatasetSchema`, batch I/O, MEBT schema tests, and existing LEBT no-offset batch tests.

Task

Make the fixed-distribution MEBT Sobol batch format conform to one versioned, schema-driven dataset contract, without changing existing generated datasets.

The current producer writes a custom layout, while the declared MEBT schema and generic batch tools describe a richer, incompatible layout. The producer also accepts `fixed_beam_current` even though its schema declares a fixed value of `0.05`.

Requirements

1. Define a generic batch representation that supports:
   - sampled and full inputs;
   - fixed parameters;
   - optional target groups;
   - validity, completion, failure reason, and production outcomes;
   - per-field names and units from `DatasetSchema`.
2. Make the fixed-distribution MEBT producer write the generic representation for new batches. Its 19 sampled controls and fixed current must be stored as an auditable full 20D input row.
3. Preserve the existing MEBT batch reader behavior through a compatibility adapter for legacy custom-layout files. Do not rewrite or delete data.
4. Make MEBT current derive from the fixed-distribution schema. Reject a production config whose current differs from the schema value, unless a new explicitly versioned schema and dataset name are supplied.
5. Preserve the current TRACK conversion boundary and unit-verification metadata. Do not claim a verified TRACK physical unit.
6. Make consolidated MEBT artifacts retain completion, validity, failure, and current metadata needed for downstream audit.

Tests

Add tests for:

1. a newly generated dry-run MEBT batch satisfying its schema;
2. full 20D controls-plus-fixed-current reconstruction;
3. rejection of a config current inconsistent with the schema;
4. legacy MEBT batch compatibility reading;
5. consolidated artifact preservation of validity and current metadata;
6. no regression in LEBT batch read/write tests.

Do Not Do

- Do not regenerate production datasets.
- Do not silently reinterpret current units.
- Do not remove support for existing MEBT batch files.

Completion Summary

Report changed files, old/new batch layouts, current validation behavior, compatibility behavior, tests run, and migration limitations.

Source Markdown: `docs/codex_refactor_tasks/12_mebt_lattice_control_manifest.md`

Codex Prompt: MEBT Lattice-Control Manifest and Drift Detection

Read `AGENTS.md`, the MEBT Sobol producer, MEBT schema, RAON MEBT lattice files, lattice parsing code, and current MEBT production tests.

Task

Make the MEBT 19D control-vector contract explicit and detect lattice changes that would otherwise silently change feature meaning while preserving vector length.

Requirements

1. Define a versioned lattice-control manifest containing, at minimum:
   - control names in canonical order;
   - element type and stable lattice identifier where available;
   - nominal value, unit, and sampled bounds;
   - source lattice path and content hash.
2. Generate the runtime control vector from this manifest rather than relying only on traversal order and synthetic `q01`/`cav01` labels.
3. Validate that the parsed lattice exactly matches the manifest in count, order, element identity/type, and control names before sample generation.
4. Store the resolved manifest and lattice hash in batch and consolidated metadata. Record the same information in the dataset-level config and metadata files.
5. Keep generated notebook lattice JSON as an optional input convenience, not the production source of truth.
6. Provide a clear error that identifies the first mismatched control and the expected versus observed lattice identity.

Tests

Add tests for:

1. the committed MEBT lattice matching the canonical manifest;
2. a reordered or substituted element causing validation failure;
3. stable nominal vector and bounds generation;
4. manifest/hash persistence in dry-run metadata;
5. fallback from notebook JSON to committed `sclinac.dat` retaining the same manifest.

Do Not Do

- Do not change physical control ranges.

- Do not alter `scratch.#07` handling or TRACK segment ordering.
- Do not regenerate datasets.

Completion Summary

Report the canonical control order, metadata fields, mismatch protections, changed files, and tests run.

Source Markdown: `docs/codex_refactor_tasks/13_unify_state_physics_metric_reports.md`

Codex Prompt: Unify Beam-State Physics Metric Reports

Read `AGENTS.md`, `ml/metrics.py`, `ml/physics_training.py`, training summaries, publication-metric code, and Task 4/Task 5 tests.

Task

Make `compute_state_physics_metrics()` the sole implementation of derived Courant-Snyder/emittance errors and covariance-validity metrics used by plain MLP and physics-regularized MLP evaluation.

Requirements

1. Remove duplicated calculation logic from physics-regularized training in favor of the canonical metric API.
2. Ensure both model types emit the same detailed report fields for each state group: component metrics, normalized metrics, derived metrics, validity fractions, warnings, and sample counts.
3. Preserve existing `physics_metrics_<split>.json` files through a documented compatibility serializer. Do not make notebooks or prior reports fail.
4. Make publication table generation consume the canonical metrics first; retain legacy fallback only for historical artifacts.
5. Define one tolerance configuration path for strict covariance reporting and record it in the emitted metric JSON.
6. Avoid mixed-unit aggregate RMSE as a physical acceptance metric.

Tests

Add or update tests for:

1. plain and physics-regularized evaluation yielding equivalent canonical metrics for identical predictions;
2. compatibility JSON containing historical required keys;
3. invalid covariance and nonfinite derived quantity reporting;
4. JSON serialization of all report fields;
5. publication tables loading both canonical and legacy artifacts.

Do Not Do

- Do not retrain models.
- Do not change loss definitions or model architecture.
- Do not change saved metric values manually.

Completion Summary

Report canonical fields, compatibility fields, changed files, tests run, and any historical-report limitations.

Source Markdown: `docs/codex_refactor_tasks/14_package_import_boundary_cleanup.md`

Codex Prompt: Package Import Boundary Cleanup

Read `AGENTS.md`, all package `__init__.py` files, the previous circular import regression, CLI entry points, and import-related tests.

Task

Reduce eager package re-exports so importing `synapticTrack` or a low-level submodule does not transitively import unrelated analysis, data generation, ML training, visualization, or TRACK runtime code.

Requirements

1. Map the import graph around `synapticTrack`, `utils`, `data`, `simulation`, and `ml` package initializers.
2. Keep only lightweight stable public types/functions eagerly imported. Require direct submodule imports for heavy workflows, or use narrowly scoped lazy exports where compatibility requires them.
3. Prevent data modules from importing simulation package initialization and prevent simulation configuration from importing the full ML package.
4. Preserve documented public imports where practical; issue deprecation guidance for imports that must move.
5. Add a fast import smoke test with a time budget for:
   - `import synapticTrack`;
   - beam/current and schema modules;
   - MEBT schema and producer modules;
   - ML metrics module.
6. Ensure imports do not initialize TRACK execution, read datasets, or require optional plotting/training dependencies.

Tests

Add tests that exercise the listed imports in isolated subprocesses and confirm no circular import error. Run focused package, data, simulation, and ML tests.

Do Not Do

- Do not remove top-level imports without a compatibility plan.
- Do not move application behavior into `__init__.py` files.
- Do not change model or physics behavior.

Completion Summary

Report the import graph changes, compatibility decisions, import timings, changed files, tests run, and remaining heavyweight imports.

Source Markdown: `docs/codex_refactor_tasks/15_training_artifact_path_contract.md`

Codex Prompt: Training Artifact Path Contract

Read `AGENTS.md`, `ml/run_artifacts.py`, Task 4 and Task 5 trainers, staged training, model comparison, publication metrics, and relevant tests.

Task

Centralize every per-run artifact path under one versioned artifact contract.

Requirements

1. Extend `RunArtifactPaths` to include standard metrics, canonical state-physics metrics, compatibility physics metrics, plots, summaries, normalizer manifest, checkpoints, config, and timing artifacts.
2. Replace manually assembled artifact filenames in training, staged training, comparison, and publication modules with this contract.
3. Add strict discovery/loading helpers that report missing, incompatible, or legacy artifacts explicitly rather than silently substituting empty data.
4. Preserve current filenames for existing training outputs unless a versioned migration adapter is provided.
5. Make emitted manifests include artifact paths and artifact-format version.
6. Keep plain MLP and physics-regularized MLP output trees comparable.

Tests

Add tests for:

1. paths for both model types and all splits;
2. strict missing-artifact diagnostics;
3. backward-compatible loading of current run directories;
4. staged-training and publication-table use of the centralized paths.

Do Not Do

- Do not retrain models.
- Do not rename or delete existing model artifacts in place.

- Do not alter reported numerical metrics.

Completion Summary

Report the artifact manifest format, compatibility behavior, changed files, tests run, and any artifacts intentionally left legacy-only.

Source Markdown: `docs/codex_refactor_tasks/16_versioned_hdf5_dataset_compatibility.md`

Codex Prompt: Versioned HDF5 Dataset Compatibility Layer

Read `AGENTS.md`, dataset generation modules, `data/batch_io.py`, dataset schemas, input/target view helpers, and tests for LEBT and MEBT data loading.

Task

Create a versioned dataset-reader compatibility layer for the legacy LEBT generator layout, no-offset Sobol batches, and MEBT Sobol batches.

Requirements

1. Define explicit format identifiers and supported versions for each existing HDF5 layout.
2. Implement read-only adapters that expose a normalized dataset interface: schema identity, inputs, full/fixed parameters when available, target groups, units, validity, completion, failure categories, and provenance.
3. Route dataset loading and audit code through format detection rather than file-path-specific assumptions.
4. Surface unavailable fields as explicit capability limitations, never as fabricated values.
5. Keep writers unchanged in this task except for adding unambiguous format metadata where it is missing.
6. Document a future opt-in migration command, but do not migrate or rewrite existing datasets automatically.

Tests

Add fixture-based tests for each format covering detection, normalized reads, unsupported capability reporting, target/input view selection, and invalid or unknown format errors.

Do Not Do

- Do not alter production datasets.
- Do not make legacy files appear to contain state labels or fixed inputs they never stored.

- Do not change sampling, TRACK execution, or model architecture.

Completion Summary

Report supported formats, normalized interface capabilities, compatibility limitations, changed files, tests run, and the proposed opt-in migration path.

Source Markdown: `docs/codex_refactor_tasks/README.md`

Refactor Tasks

These prompts capture the physics and code refactors identified in the 2026 repository review. Tasks 01–16 have been executed in numeric order; the files remain as implementation records and regression-scope specifications. Future work should preserve the contracts established by Tasks 11–16 for MEBT batch schema/current provenance, lattice control manifests, canonical state metrics, package import boundaries, training artifacts, and HDF5 compatibility.

Task	Scope
`01_fixed_parameter_inference_contract.md`	Make full 24D LEBT inference validate fixed values, including fixed current.
`02_beam_current_unit_contract.md`	Establish and enforce one explicit current-unit boundary for TRACK.
`03_mebt_surrogate_input_schema.md`	Define a reusable MEBT dataset/surrogate schema with injected beam state.
`04_physics_training_output_scaling.md`	Make output scaling effective for physics-regularized training.
`05_unify_beam_state_physics_api.md`	Consolidate state moments, Courant-Snyder, and covariance-validity logic.
`06_legacy_physics_cleanup.md`	Repair legacy Twiss/validity paths and remove avoidable runtime assumptions.
`07_track_execution_orchestration.md`	Unify path-safe LEBT/MEBT TRACK segment execution and runtime cleanup.
`08_track_diagnostic_output_contract.md`	Validate diagnostic output alignment, cardinality, units, and physical validity.
`09_production_batch_atomicity_and_failure_tracking.md`	Make generation batches atomic, resumable, and failure-auditable.

Task	Scope
`10_versioned_artifact_and_config_loading.md`	Make versioned artifact/config discovery strict, numeric, and diagnosable.
`11_mebt_batch_schema_and_fixed_current_contract.md`	Align MEBT Sobol batches with the schema contract and make fixed-current provenance enforceable.
`12_mebt_lattice_control_manifest.md`	Version the MEBT control order and detect lattice/control semantic drift before generation.
`13_unify_state_physics_metric_reports.md`	Make canonical component, Courant-Snyder, and covariance metrics common to both MLP workflows.
`14_package_import_boundary_cleanup.md`	Reduce eager package imports and prevent circular, heavyweight import paths.
`15_training_artifact_path_contract.md`	Centralize training artifact paths, manifests, discovery, and compatibility loading.
`16_versioned_hdf5_dataset_compatibility.md`	Add explicit adapters for legacy, LEBT no-offset, and MEBT HDF5 dataset formats.

Do not retrain production models as part of these refactors unless a task explicitly requests it. Preserve dataset compatibility or provide a documented migration path.

Source Markdown: `docs/hdf5_dataset_compatibility.md`

HDF5 Dataset Compatibility

`synapticTrack.data.dataset_compat` provides read-only detection and normalized views for these supported layouts:

- `legacy_lebt_dataset_v2`
- `lebt_no_offset_sobol_batch_v1`
- `lebt_no_offset_sobol_consolidated_v1`
- `mebt_sobol_schema_batch_v2`
- `mebt_sobol_schema_consolidated_v2`
- `mebt_sobol_legacy_batch_v1`

The normalized view reports capabilities for named inputs, full and fixed parameters, beam-state targets, completion flags, and structured failure categories. A missing capability is reported explicitly; compatibility readers do not invent absent fixed parameters, labels, or state targets.

Future Migration

No dataset is migrated automatically. A future opt-in CLI is intended to use a command shaped as:

```
synapticTrack dataset migrate --input legacy.h5 --output migrated.h5 --target-format <format
```

The command must write a new file, preserve source provenance and source hash, and require the user to select the target format. Until that command exists, use `read_hdf5_dataset_compat()` for legacy and current datasets.

Source Markdown: `docs/mebt_surrogate_input_schema.md`

MEBT Surrogate Input Schema

Dataset Versions

`mebt_production_scratch07_sobol_v1` remains the valid fixed-distribution MEBT diagnostic dataset. It has 19 sampled MEBT lattice controls and a fixed beam-current metadata value. Every run starts from the reference TRACK particle distribution `RAON/MEBT_lattice/scratch.#07`. It is appropriate for studying control-to-diagnostic response around that one injected distribution, but it is not the general MEBT surrogate contract.

`mebt_general_surrogate_v1` defines the future general surrogate input:

```
19D MEBT controls + 10D injected Gaussian state -> 30D full input
                                                  + fixed beam_current
```

The sampled 29D input contains the existing 11 quadrupole gradients, four cavity fields/phases, and the following injected-state parameterization:

```
x0 [mm], xp0 [mrad], y0 [mm], yp0 [mrad],
alpha_x, beta_x [mm/mrad], emit_x [mm*mrad],
alpha_y, beta_y [mm/mrad], emit_y [mm*mrad]
```

This is an explicit physical equivalent of the established 10D state-moment convention. It maps to `x_c`, `xp_c`, `y_c`, `yp_c`, `<x^2>`, `<xx'>`, `<x'^2>`, `<y^2>`, `<yy'>`, `<y'^2>` through the Courant-Snyder covariance. Positive beta and emittance are validated, and the resulting horizontal and vertical covariance determinants must be positive.

Fixed-Distribution Control Manifest

The fixed-distribution Sobol workflow uses a versioned `mebt_lattice_control_manifest_v1` derived from `RAON/MEBT_lattice/sclinac.dat`. The manifest fixes the 19D control order, control type, segment and element location, nominal value, bounds, and source SHA-256 hash. Generation validates the parsed source lattice, or optional JSON segment convenience files, against this manifest before sampling. A mismatch reports the first divergent control rather than silently applying controls to a changed lattice.

The fixed current is `0.05` in the configured synapticTrack/TRACK conversion contract. It is fixed, not zero, and is recorded with its unit boundary and verification status in batch and consolidated metadata.

Targets and Metadata

The general schema reserves these target groups:

- `diagnostics`: 20D MEBT centroid and rms measurements at MWS1–4 and MEBT exit, in mm;
- `state_mws4`: 10D centroid/second-moment state at MWS4;
- `state_mebt_exit`: 10D centroid/second-moment state at MEBT exit.

All general batches must record validity masks and failure reasons. Current metadata uses canonical synapticTrack current values in `uA`, plus the TRACK current conversion boundary and its verification status. The local repository does not establish TRACK's physical current unit, so that boundary remains explicitly unverified.

Sampling and Injection

`MEBTGeneralSamplingBounds` keeps lattice-control bounds separate from injected-state bounds. Unit-cube samples are expanded deterministically, and every expanded injected state is validated before it can be used. A future production generator must rewrite the per-sample input distribution and `track.dat` current before segmented TRACK execution. `scratch.#07` remains the nominal reference distribution from which those sampling bounds and injection validation should be calibrated; it is not reused unchanged for all samples in the general dataset.

No existing fixed-distribution MEBT batch is rewritten. The read-only HDF5 compatibility layer recognizes both current schema batches and legacy MEBT fixed-distribution batches. Legacy files expose only fields physically stored on disk; they do not acquire reconstructed fixed-current/full-input columns, control labels, state targets, or completion flags.

Source Markdown: `docs/package_import_boundaries.md`

Package Import Boundaries

Package initializers keep lightweight stable types eagerly available and retain historical workflow exports through lazy compatibility resolution. This avoids loading analysis, TRACK execution, data generation, visualization, and ML training stacks merely because a low-level module is imported.

For new code, import workflow APIs from their owning submodule, for example:

```
from synapticTrack.simulation.no_offset_sobol_production import run_generation
from synapticTrack.ml.metrics import compute_state_physics_metrics
```

```
from synapticTrack.data.dataset_compat import read_hdf5_dataset_compat
```

Top-level and package-level historical imports remain supported where possible, but they may load the relevant workflow on first attribute access. The isolated import smoke tests enforce a five-second budget and check that low-level imports do not initialize TRACK runtime, complete ML training modules, or unrelated simulation packages.

Source Markdown: `docs/physics_regularized_output_scaling.md`

Physics-Regularized Output Scaling

Physics-regularized training keeps the model's structured output in physical coordinates. This preserves the positive-definite state head and makes existing checkpoints and `LEBTSurrogateModel.predict_physical()` backward compatible.

The train-split output normalizer is nevertheless operational. The data loss is computed from normalized prediction and target residuals, using one reusable per-component `DifferentiableOutputTransform`. This balances diagnostics and state components with disparate numerical scales. The transform is fitted only on the training split and recorded in the run normalizer manifest and checkpoint metadata.

Courant-Snyder target loss, covariance positive-definiteness penalty, diagnostic rms-positivity penalty, reported metrics, and prediction APIs use physical units. Therefore their loss weights retain physical interpretation; output normalization does not silently rescale the beam-physics constraints.

The transform version is `train_split_output_transform_v1`. Existing production checkpoints without this manifest field remain physical-output checkpoints and continue to load unchanged. No existing model is reinterpreted or retrained.

Source Markdown: `docs/training_artifact_contract.md`

Training Artifact Contract

Each plain MLP and physics-regularized MLP run uses the `training_run_artifacts_v2` contract from `synapticTrack.ml.run_artifacts`. The contract preserves established filenames and adds `artifact_manifest.json` for explicit discovery.

Standard Paths

The manifest records checkpoints, configuration, normalizer manifest, timing, training history, plots, and per-split artifacts for `train`, `val`, and `test`:

- `metrics_<split>.json` for standard model metrics;
- `state_physics_metrics_<split>.json` for canonical state-physics reports;
- `physics_metrics_<split>.json` for historical compatibility reports;

- `derived_physics_metrics_<split>.json` and `covariance_validity_metrics_<split>.json` for focused table consumers.

`discover_run_artifacts()` distinguishes a valid current manifest, a supported legacy run directory with current filenames but no manifest, and invalid or unsupported manifests. `load_run_json_artifact()` returns structured status for missing or malformed files rather than treating missing metrics as valid empty data.

New training code must use `run_artifacts(run_dir)`. Existing model directories are not renamed or migrated in place.

References

Courant, E. D., and H. S. Snyder. 1958. "Theory of the Alternating-Gradient Synchrotron." *Annals of Physics* 3 (1): 1–48. https://doi.org/10.1016/0003-4916(58)90012-5.

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. "Array Programming with NumPy." *Nature* 585: 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Ostroumov, P. N., and V. N. Aseev. 2006. "TRACK: The New Beam Dynamics Code." In *Proceedings of the 2006 International Linear Accelerator Conference*.

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, et al. 2019. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." In *Advances in Neural Information Processing Systems*. Vol. 32.

Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, et al. 2011. "Scikit-Learn: Machine Learning in Python." *Journal of Machine Learning Research* 12: 2825–30.

Raffin, Antonin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. "Stable-Baselines3: Reliable Reinforcement Learning Implementations." In *Journal of Machine Learning Research*, 22:1–8. 268.

Sobol', I. M. 1967. "On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals." *USSR Computational Mathematics and Mathematical Physics* 7 (4): 86–112. https://doi.org/10.1016/0041-5553(67)90144-9.

team, The pandas development. 2020. "Pandas-Dev/Pandas: Pandas." In *Zenodo*. https://doi.org/10.5281/zenodo.3509134.

Towers, Mark, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, et al. 2024. "Gymnasium: A Standard Interface for Reinforcement Learning Environments." *arXiv Preprint arXiv:2407.17032*.

Virtanen, Pauli, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler

Reddy, David Cournapeau, Evgeni Burovski, et al. 2020. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." *Nature Methods* 17: 261–72. https://doi.org/10.1038/s41592-019-0686-2.

Wiedemann, Helmut. 2015. *Particle Accelerator Physics*. 4th ed. Springer. https://doi.org/10.1007/978-3-319-18317-6.